



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y
Tecnología

Máster Universitario en Ciberseguridad

**EVMAudit: Librería multi-
herramienta para la detec-
ción automatizada de vulne-
rabilidades en contratos in-
teligentes de Ethereum**

Trabajo Fin de Estudios

presentado por: Adrián Moreno Martín, Daniel Rovira Martínez, Paula Suárez Prieto

Dirigido por: Friman Sánchez Castaño

Ciudad: Madrid

Fecha: Curso 2025-2026

0.1. Resumen

La seguridad de los contratos inteligentes se ha convertido en un aspecto crítico dentro del ecosistema blockchain debido al elevado impacto económico que pueden provocar las vulnerabilidades presentes en este tipo de software. Aunque existen herramientas especializadas para su análisis, como Slither, Mythril o Echidna, sus resultados suelen presentarse de forma heterogénea y con dificultades para correlacionar y priorizar los hallazgos obtenidos. En este Trabajo Fin de Máster se presenta EVMAudit, una librería desarrollada en Python orientada a la ejecución conjunta de múltiples herramientas de análisis de seguridad para contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM). La solución propuesta incorpora mecanismos de normalización, correlación y priorización de resultados, permitiendo ofrecer una visión unificada y más estructurada de las vulnerabilidades detectadas. Además, la herramienta se distribuye mediante PyPI y dispone de una infraestructura de despliegue y automatización basada en Docker y CI/CD.

Palabras clave: contratos inteligentes, Ethereum, ciberseguridad, análisis de vulnerabilidades, blockchain

0.2. Abstract

Smart contract security has become a critical concern within the blockchain ecosystem due to the significant economic impact caused by software vulnerabilities. Although several specialized analysis tools, such as Slither, Mythril, and Echidna, are available, their results are often heterogeneous and difficult to correlate and prioritize. This Master's Thesis presents EVMAudit, a Python library designed to orchestrate multiple security analysis tools for Ethereum Virtual Machine (EVM) compatible smart contracts. The proposed solution includes mechanisms for result normalization, correlation, and prioritization, providing a unified and more structured view of the detected vulnerabilities. Furthermore, the tool is distributed through PyPI and includes an automated deployment infrastructure based on Docker and CI/CD practices.

Keywords: smart contracts, Ethereum, cybersecurity, vulnerability analysis, blockchain

Índice de Contenidos

0.1. Resumen	I
0.2. Abstract	I
Mecanismos de coordinación empleados	v
0.3. Organización del trabajo en grupo	v
0.3.1. Partes que aborda el TFE, distribución y estructura de la memoria .	v
0.3.2. Mecanismos de coordinación empleados	v
1. Introducción	1
1.1. 1. Introducción	1
1.1.1. 1.1. Motivación	2
1.1.2. 1.2. Planteamiento del problema	3
1.1.3. 1.3. Estructura del trabajo	4
2. Estado del arte	5
2.1. 2. Estado del arte	5
2.1.1. 2.1. Fundamentos blockchain y contratos inteligentes	5
2.1.2. 2.3. Interoperabilidad blockchain: bridges y protocolos cross-chain .	10
2.1.3. 2.4. Vulnerabilidades de seguridad en contratos inteligentes de Ethe- reum	14
2.1.4. 2.5. Ataques Reales	17
2.1.5. 2.6. Herramientas de Análisis	20
2.1.6. 2.7. Síntesis y limitaciones del estado del arte	27
3. Objetivos concretos y metodología de trabajo	28
3.1. 3. Objetivos concretos y metodología de trabajo	28
3.1.1. 3.1. Objetivo general	28
3.1.2. 3.2. Objetivos específicos	28
3.1.3. 3.3. Metodología del trabajo	29
4. Desarrollo específico de la contribución	30
4.1. 4. Desarrollo de la solución propuesta	30
4.2. 4.1. Requisitos	31

4.2.1.	4.1. Identificación de requisitos	31
4.3.	4.2.Arquitectura	33
4.3.1.	4.2. Arquitectura general de EVMAudit	33
4.4.	4.3.Runner	35
4.4.1.	4.3. Módulo Runner	35
4.5.	4.4.Normalizer	39
4.5.1.	4.4. Módulo de normalización	39
4.6.	4.5.Correlator	42
4.6.1.	4.5. Módulo de correlación	42
4.7.	4.6.SWC Catalog	46
4.7.1.	4.6. Catálogo de vulnerabilidades SWC	46
4.8.	4.7.Echidna adaptar	48
4.8.1.	4.7. Adaptador para Echidna y generación automática de propiedades	48
4.9.	4.8.Gestion de errores	52
4.9.1.	4.8. Gestión de errores y robustez de la solución	52
4.10.	4.9.Integración y distribución	55
4.10.1.	4.9. Distribución e integración de la solución	55
4.11.	4.10. Flujo completo	58
4.11.1.	4.10. Flujo completo del análisis	58
4.12.	4.11.Evaluación experimental	61
4.12.1.	4.11. Flujo completo del análisis	61
5.	Conclusiones y trabajo futuro	69
5.1.	5. Conclusiones y trabajo futuro	69
5.1.1.	5.1 Conclusiones	69
5.1.2.	5.2 Limitaciones de la solución	69
5.1.3.	5.3 Líneas de trabajo futuro	69
A.	Ejemplos de vulnerabilidades	71
A.1.	ANEXO A. Ejemplos de vulnerabilidades en contratos inteligentes	71
A.1.1.	ANEXO A.1. Vulnerabilidades técnicas de ejecución	71
A.1.2.	ANEXO A.2. Vulnerabilidades técnicas de control y privilegios	73
A.1.3.	ANEXO A.3. Vulnerabilidades económicas y del entorno	75
A.1.4.	ANEXO A.4. Errores lógicos de negocio	76

A.1.5. ANEXO A.5. Resumen de vulnerabilidades	78
B. Entorno virtual Python	80
B.1. ANEXO B. Entorno virtual Python	80
B.1.1. ANEXO B.1. Tipos de entornos virtuales	80
B.1.2. ANEXO B.2. Herramienta a usar: uv	81
C. Distribución mediante PyPI	84
C.1. ANEXO C. DISTRIBUCIÓN Y PUBLICACIÓN EN EL REGISTRO DE PAQUETES PYPI	84
C.1.1. ANEXO C.2. Proceso de Compilación del Paquete	85
C.1.2. ANEXO C.3. Seguridad y Autenticación en la Publicación	86
C.1.3. ANEXO C.4. Ejecución del Despliegue	86
C.1.4. ANEXO C.5. Ciclo de Mantenimiento y Actualización de Versiones .	87
D. Docker	88
D.1. Anexo D Docker	88
D.1.1. ANEXO D.1. CONTENEDORIZACIÓN E INFRAESTRUCTURA DE DESPLIEGUE (DOCKER)	88
D.1.2. ANEXO D.2. Fundamentos de Contenedorización: Docker y Docker Compose	88
D.1.3. ANEXO D.3. Estrategia de Construcción de la Imagen (Dockerfile) .	89
D.1.4. ANEXO D.4. Orquestación de Servicios (Docker Compose)	90
D.1.5. ANEXO D.5. Flujo de Despliegue y Ciclo de Vida del Contenedor .	90
E. Aplicación web	92
F. CI/CD	93
F.1. ANEXO F. ENTORNO DE INTEGRACIÓN CONTINUA (CI/CD) Y PU- BLICACIÓN AUTOMATIZADA	93
F.1.1. ANEXO F.1. Arquitectura del Workflow y Disparadores (Triggers) .	93
F.1.2. ANEXO F.2. Desglose Técnico de las Etapas del Pipeline	94
G. Railway	96
G.1. ANEXO G. DESPLIEGUE DE LA INFRAESTRUCTURA EN LA NUBE (RAILWAY)	96

G.1.1. ANEXO G.1. Aprovisionamiento y Configuración del Entorno Cloud	96
G.1.2. ANEXO G.2. Limitaciones de Hardware y el Problema del Desbordamiento de Memoria (OOM)	97
G.1.3. ANEXO G.3. Optimización del Sistema en Tiempo de Ejecución (RTS) de Haskell	97

Adrián Moreno Martín, Daniel Rovira Martínez, Paula Suárez Prieto
Máster Universitario en Ciberseguridad

Índice de Ilustraciones

Adrián Moreno Martín, Daniel Rovira Martínez, Paula Suárez Prieto
Máster Universitario en Ciberseguridad

Índice de Tablas

Mecanismos de coordinación empleados

0.3. Organización del trabajo en grupo

0.3.1. Partes que aborda el TFE, distribución y estructura de la memoria

Organización del trabajo en grupo - Desarrollo de la memoria

[[@ @p() * 0.5000 @p() * 0.5000@ Apartado de la memoria

Responsables

Introducción Alumno 1, Alumno 2 y Alumno 3

Estado del arte Alumno 1, Alumno 2 y Alumno 3

Objetivos y metodología de trabajo Alumno 1, Alumno 2 y Alumno 3

Desarrollo específico de la contribución: parte individual 1 Alumno 1

Desarrollo específico de la contribución: parte individual 2 Alumno 2

Desarrollo específico de la contribución: parte individual 3 Alumno 3

Desarrollo específico de la contribución: parte 4 Alumno 1, Alumno 2 y Alumno 3

Conclusiones y trabajo futuro Alumno 1, Alumno 2 y Alumno 3

0.3.2. Mecanismos de coordinación empleados

Con el objetivo de garantizar un seguimiento continuo del trabajo y mantener la coordinación entre los miembros del grupo, se estableció una reunión interna semanal en la que cada integrante exponía las tareas realizadas durante el periodo correspondiente y las posibles dificultades encontradas. Estas reuniones permitían, además, debatir las siguientes líneas de trabajo y planificar las tareas a desarrollar en las semanas posteriores.

Como herramientas de comunicación y colaboración se utilizaron principalmente WhatsApp para la comunicación diaria, Microsoft Teams para la realización de reuniones telemáticas, un repositorio compartido en GitHub para la gestión y sincronización del código fuente, y un documento compartido de Microsoft Word para la elaboración conjunta de la memoria.

Adicionalmente, tras cada una de las entregas parciales previstas en la planificación

del Trabajo Fin de Máster, se mantuvo una reunión de seguimiento con el director del trabajo. Estas sesiones permitieron revisar el progreso realizado, recibir retroalimentación sobre los resultados obtenidos y definir las acciones necesarias para las siguientes fases del proyecto.

Adrián Moreno Martín, Daniel Rovira Martínez, Paula Suárez Prieto
Máster Universitario en Ciberseguridad

1. Introducción

1.1. 1. Introducción

La tecnología blockchain ha evolucionado significativamente desde la aparición de Bitcoin en 2008 como sistema de dinero electrónico descentralizado. Con la llegada de plataformas como Ethereum, blockchain dejó de utilizarse únicamente para la transferencia de activos digitales y pasó a convertirse en una infraestructura capaz de ejecutar aplicaciones descentralizadas mediante contratos inteligentes. Estos contratos permiten automatizar lógica de negocio y gestionar activos sin necesidad de intermediarios, lo que ha impulsado el crecimiento de sectores como las finanzas descentralizadas (DeFi), los sistemas de tokenización y los protocolos de interoperabilidad blockchain.

Sin embargo, este nuevo paradigma también introduce importantes desafíos desde el punto de vista de la ciberseguridad. Los contratos inteligentes operan en entornos públicos y adversariales, donde el código es accesible para cualquier usuario y los errores pueden ser analizados y explotados por actores maliciosos. Además, la inmutabilidad de la blockchain dificulta la corrección de vulnerabilidades una vez desplegados los contratos, aumentando el impacto potencial de cualquier fallo de seguridad.

Durante los últimos años, múltiples incidentes han demostrado las consecuencias reales de estas vulnerabilidades. Ataques como The DAO, Poly Network, bZx o Euler Finance provocaron pérdidas económicas de cientos de millones de dólares y evidenciaron que los errores en contratos inteligentes no solo afectan al software, sino también a activos digitales con valor económico directo.

En este contexto, el análisis de seguridad de contratos inteligentes se ha convertido en una de las áreas más relevantes dentro de la ciberseguridad blockchain. Actualmente existen distintas herramientas especializadas que permiten detectar vulnerabilidades mediante técnicas como análisis estático, ejecución simbólica o fuzzing. No obstante, estas soluciones presentan limitaciones importantes, entre las que destacan la elevada tasa de falsos positivos, la fragmentación de resultados y la dificultad para priorizar riesgos de manera efectiva.

El presente Trabajo Fin de Máster se centra en el diseño e implementación de una librería en Python orientada al análisis de seguridad de contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM). La propuesta busca integrar distintas herramien-

tas de análisis existentes en una arquitectura modular capaz de correlacionar hallazgos, normalizar resultados heterogéneos y facilitar una evaluación más útil y estructurada de las vulnerabilidades detectadas.

La finalidad del trabajo no es reemplazar las herramientas actuales, sino proporcionar una capa adicional de correlación y análisis que mejore la interpretación de resultados y reduzca algunas de las limitaciones presentes en el estado del arte. Para ello, se estudiarán las principales técnicas de auditoría de smart contracts, las vulnerabilidades más relevantes en Solidity y las herramientas más utilizadas en entornos profesionales y académicos.

Finalmente, el trabajo incluirá el diseño de la arquitectura de la solución propuesta, su implementación como librería software y una evaluación experimental utilizando contratos vulnerables conocidos y casos reales de código abierto.

1.1.1. 1.1. Motivación

La auditoría de contratos inteligentes se ha convertido en una necesidad crítica dentro del ecosistema blockchain debido al creciente volumen económico gestionado por aplicaciones descentralizadas y protocolos DeFi. En este contexto, la detección temprana de vulnerabilidades resulta fundamental para reducir riesgos de explotación y minimizar pérdidas económicas.

Actualmente existen diversas herramientas orientadas al análisis automatizado de contratos inteligentes, como Slither, Mythril o Echidna, que aplican técnicas de análisis diferentes y complementarias. Sin embargo, su utilización conjunta continúa presentando dificultades relevantes. Cada herramienta genera resultados con formatos, niveles de detalle y criterios de severidad distintos, lo que dificulta la correlación de hallazgos y obliga a realizar revisiones manuales adicionales durante el proceso de auditoría.

Además, muchas soluciones actuales producen un elevado número de falsos positivos o detectan vulnerabilidades redundantes, complicando la priorización efectiva de riesgos. Esta situación provoca que los auditores deban invertir una cantidad significativa de tiempo en interpretar resultados y validar manualmente la relevancia real de cada hallazgo.

La motivación principal de este trabajo surge de la necesidad de mejorar este proceso mediante una solución que permita integrar y normalizar información procedente de múltiples herramientas de análisis. Se busca proporcionar una visión más estructurada y útil del estado de seguridad de un contrato inteligente, facilitando tanto la interpretación de resultados como la identificación de vulnerabilidades relevantes.

Desde el punto de vista académico y profesional, el trabajo también pretende contribuir al estudio de técnicas de análisis aplicadas a contratos inteligentes y explorar enfoques que permitan mejorar la automatización de auditorías de seguridad en entornos blockchain.

1.1.2. 1.2. Planteamiento del problema

El análisis de seguridad de contratos inteligentes presenta dificultades específicas derivadas tanto del modelo de ejecución de blockchain como de las limitaciones de las herramientas actuales de auditoría automática.

Por un lado, los contratos inteligentes operan en un entorno especialmente adversarial. El código fuente y el bytecode suelen ser públicos, las transacciones son observables y los atacantes pueden estudiar el comportamiento interno de los contratos antes de explotar una vulnerabilidad. Además, la ejecución determinista de la Ethereum Virtual Machine y la inmutabilidad del despliegue implican que muchos errores no puedan corregirse fácilmente una vez publicados en la red.

Por otro lado, las herramientas automáticas de análisis existentes presentan limitaciones relevantes. Soluciones ampliamente utilizadas como Slither, Mythril o Echidna aplican técnicas diferentes y generan resultados heterogéneos tanto en formato como en nivel de detalle. Esto provoca problemas como:

- Detección redundante de una misma vulnerabilidad por múltiples herramientas.
- Generación de falsos positivos.
- Dificultad para correlacionar hallazgos relacionados.
- Ausencia de criterios homogéneos de priorización.
- Escasa contextualización del impacto real de las vulnerabilidades.

Como consecuencia, los procesos de auditoría continúan dependiendo en gran medida de la revisión manual realizada por expertos, especialmente para validar resultados y diferenciar vulnerabilidades críticas de hallazgos con impacto reducido.

Ante esta situación, el problema abordado en este trabajo consiste en cómo mejorar el análisis automatizado de contratos inteligentes mediante un sistema capaz de integrar múltiples herramientas de seguridad, normalizar sus resultados y proporcionar una visión más estructurada y útil de las vulnerabilidades detectadas.

Para ello, se propone el desarrollo de una librería en Python orientada a la correlación de hallazgos de seguridad en contratos inteligentes compatibles con la EVM, utilizando un enfoque modular que facilite la integración de distintas técnicas de análisis.

1.1.3. 1.3. Estructura del trabajo

El presente documento se organiza en varios capítulos que permiten abordar de forma progresiva tanto el contexto teórico del problema como el desarrollo de la solución propuesta.

En primer lugar, el capítulo de introducción presenta el contexto general del trabajo, la motivación del estudio y el problema identificado.

A continuación, el capítulo correspondiente al estado del arte recoge los fundamentos necesarios para comprender el análisis de seguridad en contratos inteligentes. En este apartado se abordan conceptos relacionados con blockchain, Ethereum, contratos inteligentes, así como las principales vulnerabilidades de seguridad, técnicas de auditoría y herramientas de análisis existentes en la actualidad.

Posteriormente, el capítulo de objetivos concretos y metodología de trabajo define el objetivo general del TFM, los objetivos específicos y las distintas fases planteadas para el desarrollo de la solución propuesta. También se describe el enfoque metodológico utilizado para la investigación, diseño, implementación y validación del sistema.

El capítulo de desarrollo específico de la contribución constituye el núcleo principal del trabajo. En él se describe el diseño e implementación de la librería desarrollada, incluyendo la arquitectura del sistema, los módulos principales, la integración de herramientas externas, el proceso de normalización y correlación de resultados y los mecanismos de evaluación empleados.

Finalmente, el capítulo de conclusiones y trabajo futuro recoge las principales aportaciones del trabajo, las limitaciones identificadas durante el desarrollo y posibles líneas de mejora o evolución futura de la herramienta propuesta.

2. Estado del arte

2.1. 2. Estado del arte

El presente capítulo recoge los fundamentos técnicos y el contexto necesario para comprender el problema abordado en este Trabajo Fin de Máster. En primer lugar, se introducen los conceptos básicos de blockchain y contratos inteligentes, con especial atención a Ethereum y al modelo de ejecución de la Ethereum Virtual Machine. Posteriormente, se analizan los principales retos de seguridad asociados a los contratos inteligentes, las vulnerabilidades más frecuentes, los ataques reales más representativos y las herramientas actuales de análisis. Finalmente, se identifican las limitaciones del estado del arte que justifican el desarrollo de una librería en Python orientada a la integración y correlación de resultados de seguridad.

2.1.1. 2.1. Fundamentos blockchain y contratos inteligentes

La tecnología **blockchain** se define como un sistema de registro distribuido (*distributed ledger*) que permite almacenar y gestionar información de forma descentralizada, segura y resistente a manipulaciones, sin depender de una autoridad central de confianza. Este paradigma fue introducido por Satoshi Nakamoto en 2008, en el contexto del diseño de Bitcoin, donde se propone un sistema de dinero electrónico *peer-to-peer* basado en un libro mayor compartido entre múltiples nodos de la red. [1]

En una blockchain, las transacciones se agrupan en estructuras denominadas **bloques**, que se enlazan secuencialmente mediante **funciones hash criptográficas**. Cada bloque contiene, entre otros elementos, un conjunto de transacciones validadas, una marca temporal y el hash del bloque anterior. Este encadenamiento garantiza la integridad del sistema, ya que cualquier modificación en un bloque previamente confirmado alteraría su hash, rompiendo la continuidad de la cadena y permitiendo detectar manipulaciones de forma inmediata. [2]

Figura 1. Cadena genérica de bloques

Fuente: Blockchain Technology Overview, NISTIR 8202.

Adicionalmente, blockchain emplea criptografía asimétrica para asegurar la autenticidad e integridad de las transacciones. Cada participante dispone de un par de claves criptográficas (clave pública y clave privada), donde la clave privada se utiliza para firmar

digitalmente las transacciones, y la clave pública permite a otros nodos verificar su validez. Este mecanismo elimina la necesidad de intermediarios de confianza, trasladando la verificación al propio sistema distribuido. [2]

2.1.1. Tipos de redes blockchain

Las redes blockchain pueden clasificarse en función de su modelo de acceso y gobernanza, distinguiéndose principalmente entre redes públicas (*permissionless*) y redes privadas (*permissioned*).

Las ***blockchains permissionless*** permiten la participación abierta de cualquier usuario, que puede leer el estado del *ledger* y, dependiendo del protocolo de consenso, participar en la validación de bloques. Este modelo prioriza la descentralización, la transparencia y la resistencia a la censura. Ejemplos representativos incluyen Bitcoin y Ethereum.

Por otro lado, las ***blockchains permissioned*** restringen el acceso a un conjunto predefinido de participantes autorizados. En este contexto, los nodos validadores son conocidos y controlados, lo que permite optimizar el rendimiento, reducir el coste computacional y facilitar el cumplimiento de requisitos regulatorios. Sin embargo, este modelo introduce un mayor grado de centralización. Un ejemplo ampliamente adoptado en entornos empresariales es Hyperledger Fabric.

En términos generales, las redes públicas priorizan la transparencia y la descentralización, mientras que las redes privadas favorecen la eficiencia operativa, el control y la privacidad, lo que explica su adopción en contextos corporativos. [2]

2.1.2. Consenso entre nodos

Uno de los problemas fundamentales en sistemas distribuidos consiste en lograr que un conjunto de nodos alcance un acuerdo sobre el estado del sistema, incluso en presencia de fallos o comportamientos maliciosos. Este problema ha sido ampliamente estudiado en la literatura, destacando el trabajo de Leslie Lamport junto con Shostak y Pease, quienes formalizaron el consenso en entornos con fallos arbitrarios mediante el *Byzantine Generals Problem*, sentando las bases de la tolerancia a fallos bizantinos (*Byzantine Fault Tolerance*, BFT). [3]

En este contexto, un sistema tolerante a fallos bizantinos debe ser capaz de alcanzar consenso incluso cuando algunos nodos presentan comportamientos maliciosos o arbitrarios, como el envío de información falsa o la manipulación deliberada del protocolo. Este

modelo resulta especialmente relevante en redes abiertas y sin permisos, como las blockchain públicas, donde no existe una relación de confianza previa entre los participantes.

No obstante, no todos los algoritmos de consenso contemplan este modelo adversarial. Protocolos clásicos como *Paxos Protocol* están diseñados para entornos con fallos por parada (*crash faults*), en los que los nodos pueden fallar o dejar de responder, pero no actúan de forma maliciosa. Esta limitación los hace inadecuados para escenarios abiertos como blockchain, donde se asume la posible existencia de actores maliciosos. [4]

A partir de estos fundamentos, las redes blockchain han desarrollado mecanismos de consenso específicos que permiten mantener la coherencia del sistema en entornos descentralizados y potencialmente hostiles. [2] Entre los más relevantes destacan:

- **Proof of Work (PoW)**: basado en la resolución de problemas criptográficos que requieren un elevado coste computacional. Este mecanismo, utilizado por Bitcoin, proporciona seguridad al hacer económicamente inviable la manipulación de la cadena, aunque presenta limitaciones en términos de eficiencia energética y escalabilidad.
- **Proof of Stake (PoS)**: sustituye el esfuerzo computacional por la participación económica, donde los validadores son seleccionados en función de la cantidad de activos bloqueados (*stake*). Este enfoque mejora la eficiencia energética, pero introduce nuevos riesgos, como la concentración de poder o ataques derivados del problema *nothing-at-stake*.
- **Delegated Proof of Stake (DPoS)**: variante de PoS en la que los validadores son elegidos mediante mecanismos de votación. Aumenta el rendimiento del sistema, aunque reduce el grado de descentralización.
- **Proof of Authority (PoA)**: modelo en el que un conjunto limitado de nodos autorizados valida las transacciones. Se utiliza principalmente en blockchains privadas, donde la identidad de los participantes es conocida.

Cada uno de estos mecanismos implica distintos compromisos entre seguridad, descentralización y rendimiento, lo que en la literatura se describe frecuentemente como un conjunto de *trade-offs* inherentes al diseño de sistemas blockchain, comúnmente denominado el “trilema de blockchain”. [5], [6]

Desde una perspectiva de ciberseguridad, la elección del protocolo de consenso no solo condiciona la arquitectura del sistema, sino también su superficie de ataque. En particular,

estos compromisos influyen directamente en la viabilidad de vectores de explotación como ataques del 51 %, censura de transacciones o manipulación del orden de ejecución (*front-running*), especialmente en entornos abiertos y sin permisos.

2.1.3. Fundamentos de contratos inteligentes

Los **contratos inteligentes** (*smart contracts*) constituyen una extensión funcional de la tecnología blockchain que permite la ejecución de lógica programable de forma descentralizada. El concepto fue introducido por Nick Szabo en 1994, quien los definió como “un protocolo de transacción informatizado que ejecuta los términos de un contrato” [7]

En la actualidad, los contratos inteligentes se materializan como programas desplegados en una red blockchain que encapsulan tanto lógica como estado persistente. Su ejecución es llevada a cabo por los nodos de la red, los cuales deben alcanzar un resultado determinista e idéntico para garantizar la coherencia global del sistema. Este requisito impone restricciones relevantes en el diseño del software, especialmente en lo relativo al uso de fuentes externas de información o funciones no deterministas.

Cabe destacar que no todas las plataformas blockchain soportan contratos inteligentes. Redes como Bitcoin incorporan un lenguaje de scripting limitado, diseñado para priorizar la seguridad y la simplicidad, mientras que otras plataformas como Ethereum permiten la ejecución de código arbitrario, habilitando así el desarrollo de aplicaciones complejas sobre la blockchain. [8]

2.1.4. Ethereum Virtual Machine

La adopción generalizada de contratos inteligentes se consolidó con la aparición de Ethereum, que introduce un entorno de ejecución específico denominado **Ethereum Virtual Machine (EVM)**. Esta máquina virtual permite la ejecución de código de propósito general dentro de un entorno distribuido y determinista.

El funcionamiento interno de Ethereum se describe formalmente en el *Yellow Paper*, elaborado por Gavin Wood, donde se definen:

- La arquitectura y semántica de la EVM
- El modelo de ejecución de contratos y transacciones
- El sistema de gas como mecanismo de control de recursos

La EVM es una máquina virtual basada en pila (*stack-based*), donde cada operación tiene un coste asociado en gas. Este mecanismo permite limitar el consumo de recursos computacionales y actúa como protección frente a ataques de denegación de servicio, al imponer un coste económico a la ejecución de operaciones complejas o potencialmente abusivas. [9]

2.1.5. Propiedades de los contratos inteligentes

Desde un punto de vista técnico, los contratos inteligentes presentan una serie de propiedades fundamentales[2]:

- **Determinismo:** la ejecución debe producir el mismo resultado en todos los nodos, garantizando la coherencia del sistema distribuido.
- **Inmutabilidad:** una vez desplegados, los contratos no pueden modificarse directamente, lo que dificulta la corrección de errores.
- **Transparencia:** en redes públicas, el código y el estado del contrato son accesibles, favoreciendo la auditabilidad.
- **Ejecución descentralizada:** no existe un punto único de control, eliminando dependencias de entidades centrales.

2.1.6. Limitaciones y riesgos

A pesar de sus ventajas, los contratos inteligentes presentan importantes desafíos desde el punto de vista de la ciberseguridad [2]:

- **Vulnerabilidades en el código:** errores de implementación pueden derivar en fallos críticos explotables.
- **Problemas de control de acceso:** una gestión incorrecta de permisos puede permitir operaciones no autorizadas.
- **Dependencia de oráculos externos:** introduce confianza en terceros y posibles vectores de manipulación.
- **Inmutabilidad del código:** dificulta la corrección de vulnerabilidades tras el despliegue.

- **Costes de ejecución:** el modelo de gas puede ser explotado para provocar condiciones de denegación de servicio.

Estas características hacen que los errores en contratos inteligentes puedan tener consecuencias críticas, como la pérdida irreversible de activos digitales. [2], [10]

2.1.7. Aplicaciones descentralizadas (DApps)

Sobre la base del modelo de ejecución, las propiedades y las limitaciones descritas, los contratos inteligentes permiten la construcción de aplicaciones descentralizadas (*Decentralized Applications*, DApps), que constituyen sistemas completos en los que la lógica de negocio se implementa parcial o totalmente mediante contratos inteligentes. [11]

A diferencia de las aplicaciones tradicionales basadas en arquitecturas cliente-servidor, las DApps distribuyen la lógica entre múltiples nodos de la red, eliminando intermediarios y reduciendo la dependencia de entidades centralizadas. Este cambio implica una redefinición del modelo de confianza, donde los usuarios depositan su confianza en el código ejecutado en la blockchain y en las garantías del protocolo subyacente.

Desde la perspectiva de la ciberseguridad, este paradigma introduce implicaciones relevantes. La transparencia del código facilita su auditoría, pero también permite a actores maliciosos analizarlo en busca de vulnerabilidades. Asimismo, la composición de múltiples contratos y la dependencia de servicios externos amplían la superficie de ataque, pudiendo dar lugar a vulnerabilidades complejas en sistemas interconectados, como ocurre en el ecosistema DeFi.

Además, la inmutabilidad de los contratos implica que los errores no pueden corregirse fácilmente tras su despliegue, lo que incrementa el impacto potencial de cualquier vulnerabilidad explotada. [12]

2.1.2. 2.3. Interoperabilidad blockchain: bridges y protocolos cross-chain

2.3.1. El problema de las blockchains aisladas

El ecosistema blockchain no es un sistema unificado, sino un conjunto de redes independientes con arquitecturas, lenguajes de contrato, modelos de consenso y activos nativos distintos. Ethereum concentra la mayor parte de la liquidez DeFi, pero redes como BNB Chain, Solana, Avalanche, Polygon o Arbitrum cuentan con ecosistemas propios de considerable tamaño e importancia económica. Esta fragmentación responde en parte a la

búsqueda de propiedades específicas (mayor rendimiento, menores costes de transacción o mayor privacidad) que ninguna cadena única ha logrado ofrecer de forma simultánea con suficiente madurez [15].

Sin embargo, la coexistencia de múltiples redes introduce un problema estructural: por diseño, una blockchain es un sistema cerrado que opera bajo sus propias reglas de consenso y no tiene capacidad para verificar de forma nativa el estado de otra cadena [15]. Un nodo de Ethereum no puede comprobar directamente si una transacción ha ocurrido en Solana ni si un activo ha sido bloqueado en BNB Chain. Esta limitación, conocida en la literatura como el problema de la comunicación entre *ledgers* distribuidos, impide que los activos y la información fluyan libremente entre redes sin recurrir a mecanismos intermediarios [15][16]

Las consecuencias prácticas de esta fragmentación son significativas. La liquidez se encuentra dispersa entre múltiples cadenas, lo que encarece las operaciones para los usuarios y reduce la eficiencia del mercado. Los usuarios que desean aprovechar oportunidades en distintas redes se ven obligados a recurrir a *exchanges* centralizados o a mecanismos de transferencia que introducen costes, demoras y, en muchos casos, riesgos adicionales. En respuesta a esta problemática, la industria ha desarrollado una categoría de protocolos conocidos como bridges o puentes cross-chain, cuya función es permitir la transferencia de activos e información entre blockchains heterogéneas [15][16].

Zamyatin et al. formalizan este problema en su trabajo seminal sobre comunicación entre ledgers distribuidos, estableciendo que cualquier protocolo de interoperabilidad debe garantizar propiedades de seguridad como la atomicidad (la transferencia ocurre por completo o no ocurre) y la consistencia (el estado de ambas cadenas refleja correctamente la operación realizada) [15]. Su análisis concluye que cumplir estas propiedades de forma simultánea en un entorno sin confianza requiere suposiciones criptográficas o de confianza que constituyen, precisamente, los puntos de fragilidad del sistema.

2.3.2. Tipos de bridges y arquitecturas

La literatura clasifica los bridges según dos dimensiones complementarias: el mecanismo de transferencia que utilizan y el modelo de confianza sobre el que se apoyan [15][16]. Esta clasificación es relevante no solo desde el punto de vista técnico, sino también desde la perspectiva de la seguridad, ya que distintos diseños implican distintas superficies de ataque y distintos supuestos de confianza.

Desde el punto de vista del mecanismo de transferencia, los modelos principales son cuatro.

El **modelo *Lock-and-Mint***, el más extendido en la práctica, bloquea los activos en un contrato de custodia en la cadena de origen y acuña tokens representativos en la cadena de destino. Cuando el usuario desea recuperar sus activos originales, los tokens representativos son destruidos y el contrato de custodia libera los activos bloqueados. Este modelo es sencillo de implementar, pero introduce un riesgo de concentración: el contrato de custodia en la cadena de origen se convierte en un punto crítico que concentra los activos de todos los usuarios [16].

El **modelo *Burn-and-Mint*** elimina la necesidad de custodia centralizada destruyendo los tokens en la cadena de origen antes de acuñar equivalentes en la cadena de destino. Esto requiere que el token cuente con contratos de emisión desplegados en ambas cadenas con autoridad para quemar y acuñar. Los *Atomic Swaps* permiten intercambios directos entre cadenas mediante contratos de tiempo bloqueado (*Hash Time-Locked Contracts*, HTLC), que garantizan que la transferencia es atómica sin necesidad de custodia de terceros, aunque con limitaciones de liquidez y compatibilidad entre cadenas [15]. Finalmente, las *Liquidity Networks* utilizan pools de liquidez pre-fondeados en cada cadena para facilitar las transferencias, reduciendo los tiempos de espera a costa de requerir capital inmovilizado en ambos extremos.

Desde el punto de vista del modelo de confianza, Belchior et al. establecen una taxonomía en cuatro niveles [16]. Los bridges custodiales o centralizados confían en una única entidad que custodia los activos bloqueados y autoriza las acuñaciones en la cadena destino, lo que los hace rápidos y sencillos pero equivalentes en términos de riesgo a un *exchange* centralizado. Los bridges basados en *multisig* distribuyen la autorización entre un conjunto de validadores, requiriendo que un umbral mínimo de ellos firme cada operación; su seguridad depende directamente del número de validadores, su independencia y la protección de sus claves privadas. Los bridges con *light clients* eliminan la necesidad de confiar en validadores externos verificando pruebas criptográficas del estado de la cadena de origen directamente en el contrato de la cadena de destino, lo que los hace más seguros, pero también más complejos y costosos en términos de gas. Los *ZK-bridges*, el estado del arte actual, utilizan pruebas de conocimiento cero para demostrar matemáticamente la corrección de las *transacciones cross-chain* sin revelar información adicional, ofreciendo garantías de seguridad muy superiores, si bien su adopción generalizada aún está limitada

por la complejidad computacional de generación de pruebas [16].

2.3.3. Componentes técnicos de un bridge

Con independencia del modelo concreto adoptado, los bridges comparten una serie de componentes técnicos cuya correcta implementación es determinante para su seguridad [15][16].

Los contratos inteligentes de bloqueo y acuñación constituyen el núcleo del bridge en la capa *on-chain*. En la cadena de origen, un contrato recibe y custodia los activos del usuario, emitiendo un evento que sirve como prueba de la operación. En la cadena de destino, un segundo contrato recibe la prueba verificada y acuña los activos representativos. La lógica de estos contratos debe implementar correctamente las comprobaciones de autorización, los mecanismos de verificación de pruebas y las condiciones de reversión ante situaciones anómalas [15].

Los validadores o *relayers* son los componentes que operan fuera de la cadena y cuya función es detectar eventos en la cadena de origen, construir las pruebas correspondientes y enviarlas al contrato de la cadena de destino. En *bridges multisig*, cada validador firma independientemente la prueba de la operación, y el contrato de destino verifica que se ha alcanzado el umbral de firmas requerido antes de ejecutar la acuñación. La seguridad de este componente depende en gran medida de la gestión de claves privadas de los validadores y de la distribución del conjunto validador [16].

Los sistemas de verificación de pruebas son el mecanismo mediante el cual el contrato de la cadena de destino comprueba que la operación declarada por el *relayer* ha ocurrido realmente en la cadena de origen. Según la arquitectura del bridge, esta verificación puede basarse en la comprobación de un umbral de firmas de validadores autorizados, en la verificación de una prueba Merkle sobre el estado de la cadena de origen, o en la verificación de una prueba criptográfica de conocimiento cero. La corrección de este componente es crítica: un fallo en la lógica de verificación puede permitir a un atacante fabricar pruebas falsas y acuñar activos sin respaldo [15].

Finalmente, los mecanismos de seguridad complementarios incluyen sistemas de límites de transferencia por período de tiempo, funciones de pausa de emergencia, esquemas de *timelock* para operaciones de actualización del protocolo y sistemas de monitorización *on-chain*. Aunque estos mecanismos no eliminan las vulnerabilidades subyacentes, pueden limitar el impacto de un exploit al reducir la ventana de tiempo durante la cual un atacante

puede extraer fondos antes de que el protocolo sea pausado [16].

La combinación de todos estos componentes hace que los bridges sean sistemas de complejidad técnica considerablemente superior a la de un contrato DeFi convencional. Operan simultáneamente en múltiples entornos de ejecución heterogéneos, dependen de componentes *off-chain* con sus propios vectores de compromiso, y gestionan volúmenes de activos que los convierten en objetivos de alto valor económico. Esta combinación de complejidad, heterogeneidad y concentración de activos explica por qué los bridges han concentrado algunas de las pérdidas más significativas de la historia del ecosistema blockchain, y por qué su análisis de seguridad plantea desafíos específicos que trascienden las capacidades de las herramientas de análisis de contratos inteligentes actualmente disponibles [17]. Las vulnerabilidades concretas que afectan a estos sistemas, así como los ataques reales que las han materializado, se analizan en los apartados siguientes de este estado del arte.

2.1.3. 2.4. Vulnerabilidades de seguridad en contratos inteligentes de Ethereum

Los contratos inteligentes en Ethereum presentan un modelo de seguridad distinto al del software tradicional. Su ejecución es pública, el código es accesible y cualquier error puede ser analizado y explotado por actores con incentivos económicos directos. Además, la inmutabilidad dificulta la corrección de fallos una vez desplegado el contrato, y la interacción con otros contratos y servicios externos incrementa la superficie de ataque. [10], [30]

Por este motivo, una vulnerabilidad en Solidity no debe entenderse solo como un error de programación, sino como una debilidad explotable en un entorno adversarial. En la práctica, los problemas de seguridad no se limitan a fallos técnicos, sino que también incluyen errores de diseño y de lógica de negocio. [31]

Para su análisis, resulta útil agrupar las vulnerabilidades en cuatro categorías principales:

- Vulnerabilidades técnicas de ejecución
- Vulnerabilidades de control y privilegios
- Vulnerabilidades económicas y dependencia del entorno

- Errores lógicos de negocio

2.4.1. Vulnerabilidades técnicas de ejecución

Este grupo incluye errores directamente relacionados con la ejecución en la EVM y la implementación en Solidity.

Una de las más conocidas es la **reentrancy**, que ocurre cuando un contrato realiza una llamada externa antes de actualizar su estado interno. Esto permite que el contrato receptor vuelva a ejecutar la función vulnerable con un estado inconsistente. A pesar de que existen patrones de mitigación bien conocidos, sigue siendo una de las vulnerabilidades más relevantes en Ethereum. [32]

Otra categoría importante son los **problemas aritméticos**, como overflow y underflow. Aunque Solidity 0.8 introduce comprobaciones automáticas, estos errores siguen siendo relevantes en contratos antiguos, en bloques unchecked o en cálculos financieros incorrectos.

También destacan las **llamadas externas inseguras**, ya que cualquier call transfiere el control de ejecución a otro contrato. Esto puede provocar comportamientos inesperados o ejecución de lógica maliciosa. En esta línea, el uso de delegatecall es especialmente crítico, ya que ejecuta código externo sobre el almacenamiento del contrato llamador, pudiendo comprometer completamente su estado. [30]

Por último, los **ataques de denegación de servicio (DoS)** son frecuentes en este entorno. No buscan tumbar el sistema, sino bloquear funciones críticas, por ejemplo, mediante bucles no acotados o reversiones provocadas por contratos externos. [30]

2.4.2. Vulnerabilidades de control y privilegios

Muchas vulnerabilidades reales no se deben a errores técnicos complejos, sino a problemas en el control de acceso.

Esto incluye funciones sensibles sin restricciones, roles mal definidos o privilegios excesivos. Un error típico es permitir que cualquier usuario ejecute funciones críticas como transferencias de fondos o cambios de configuración. [33]

Un caso especialmente problemático es el uso de tx.origin para autenticación. Este valor representa el origen externo de la transacción, no el llamador inmediato, lo que permite ataques mediante contratos intermedios.

También son relevantes los problemas en contratos upgradeables, especialmente los relacionados con inicializadores. Si una función inicialize no está correctamente protegida, un atacante puede ejecutarla y asumir el control del contrato. [34]

2.4.3. Vulnerabilidades económicas y dependencia del entorno

En muchos casos, el problema no está en el código en sí, sino en cómo interactúa el contrato con su entorno.

El ejemplo más claro es el **front-running**, donde un atacante observa transacciones en la mempool y envía otras con mayor prioridad para ejecutarse antes. Este fenómeno forma parte del problema más amplio del **MEV (Maximal Extractable Value)** y afecta especialmente a aplicaciones DeFi. [35]

Otro riesgo importante es la **manipulación de oráculos**. Si un contrato depende de precios externos que pueden alterarse temporalmente, un atacante puede aprovechar esa situación para obtener beneficios indebidos, por ejemplo, en protocolos de préstamo. [33]

En esta categoría también se incluyen los problemas de **aleatoriedad insegura** y el uso de `block.timestamp` como fuente de decisiones críticas. Estas variables no son completamente fiables ni impredecibles, por lo que pueden ser manipuladas o anticipadas en determinados escenarios.

2.4.4. Errores lógicos de negocio

Los errores más difíciles de detectar son los relacionados con la lógica del contrato.

En estos casos, el código puede ser correcto desde el punto de vista técnico, pero implementar una lógica incorrecta. Esto incluye errores en cálculos de balances, distribución de recompensas, gestión de estados o validación de condiciones. [31]

Este tipo de vulnerabilidades es especialmente relevante porque suele escapar a las herramientas automáticas. Además, muchos ataques reales combinan errores lógicos con otros factores, como manipulación de precios o condiciones de ejecución. [35]

2.4.5. Conclusión

En conjunto, la seguridad en contratos inteligentes no puede abordarse únicamente mediante la detección de patrones conocidos. Aunque vulnerabilidades como reentrancy o overflow siguen siendo relevantes, los problemas más complejos suelen estar relacionados con el diseño del sistema, la interacción con otros componentes y la lógica de negocio.

Esto implica que una auditoría efectiva debe analizar no solo el código, sino también su comportamiento, sus dependencias y los incentivos que lo rodean.

2.1.4. 2.5. Ataques Reales

El estudio de incidentes reales constituye una fuente de conocimiento imprescindible en el ámbito de la seguridad de contratos inteligentes. A diferencia de los entornos de prueba, los ataques producidos en redes de producción demuestran cómo las vulnerabilidades teóricas se traducen en pérdidas económicas concretas e irreversibles.

En esta sección se analizan cuatro casos históricos representativos, seleccionados por su relevancia técnica y su correspondencia directa con cada una de las categorías de vulnerabilidades descritas en la sección anterior (4.4. Vulnerabilidades de seguridad en contratos inteligentes de Ethereum).

2.5.1. Vulnerabilidad técnica de ejecución: The DAO (2016)

El ataque contra The DAO, producido en junio de 2016, representa el caso más paradigmático de explotación de una vulnerabilidad de *reentrancy* en la historia de Ethereum [36]. The DAO era un fondo de inversión descentralizado desplegado sobre Ethereum que había recaudado aproximadamente 150 millones de dólares en ETH procedentes de más de once mil participantes, convirtiéndose en uno de los mayores proyectos de financiación colectiva hasta ese momento [37].

La vulnerabilidad residía en la función `withdraw()` del contrato. El flujo de ejecución enviaba los fondos en ETH al destinatario antes de actualizar el balance interno correspondiente. Esta secuencia incorrecta permitió a un atacante desplegar un contrato receptor cuya función de *fallback* re-invocaba `withdraw()` de forma recursiva, extrayendo fondos de manera repetida mientras el saldo del contrato permanecía sin modificar. El ataque se ejecutó en múltiples iteraciones dentro de una misma cadena de llamadas, drenando aproximadamente 3,6 millones de ETH [38].

Las consecuencias del incidente trascendieron lo puramente económico. La comunidad de Ethereum debatió durante semanas sobre cómo dar respuesta al ataque, dado el carácter inmutable del protocolo. Finalmente, se ejecutó un *hard fork* en el bloque 1.920.000, el 20 de julio de 2016, que revirtió el estado de la cadena para recuperar los fondos. Una fracción de la comunidad rechazó esta intervención por considerarla contraria a los principios de inmutabilidad de la tecnología blockchain, lo que dio lugar a la bifurcación conocida como

Ethereum Classic [37].

Este caso puso de manifiesto que el patrón de diseño correcto para gestionar transferencias es el denominado *checks-effects-interactions*: primero verificar condiciones, después actualizar el estado del contrato y, por último, realizar cualquier llamada externa. Su incumplimiento en The DAO, a pesar de la aparente sencillez del principio, resultó en pérdidas valoradas en torno a 60 millones de dólares al precio del momento [36].

2.5.2. Vulnerabilidad de control y privilegios: Poly Network (2021)

El ataque a Poly Network, ocurrido el 10 de agosto de 2021, ilustra con especial claridad las consecuencias de un control de acceso deficiente en contratos que gestionan operaciones críticas de alto valor [39]. Poly Network es un protocolo de interoperabilidad *cross-chain* que conecta diversas redes blockchain, lo que amplía considerablemente su superficie de ataque.

El vector de explotación se basó en una ausencia de restricciones en la cadena de llamadas entre contratos del protocolo. La función `verifyHeaderAndExecuteTx()`, presente en el contrato `EthCrossChainManager`, permitía invocar internamente la función `PutCurrEpochConPubKeyBytes()` del contrato `EthCrossChainData`, encargada de actualizar la clave pública del grupo de *keepers* con permisos para ejecutar transacciones *cross-chain*. Al no existir ninguna comprobación que restringiese qué direcciones podían realizar esta invocación de forma indirecta, el atacante fue capaz de sustituir los *keepers* legítimos por una dirección bajo su propio control.

La operación se replicó de forma simultánea en tres redes: Ethereum, BNB Chain y Polygon; lo que supuso un control inmediato sobre la práctica totalidad de los fondos bloqueados en el protocolo, con un valor estimado de 611 millones de dólares, convirtiéndose en el mayor robo de la historia del ecosistema DeFi hasta esa fecha.

El desenlace del incidente fue inusual: el atacante devolvió la totalidad de los fondos en los días posteriores, argumentando haber actuado con el propósito de demostrar la vulnerabilidad. No obstante, el caso evidencia que errores de diseño en el modelo de permisos (independientemente de su recuperabilidad) pueden comprometer completamente la integridad de un protocolo. Una correcta implementación de control de acceso, mediante modificadores de autorización explícitos y separación de privilegios entre contratos, habría impedido la escalada de permisos aprovechada en este ataque [40].

2.5.3. Vulnerabilidad económica y dependencia del entorno: bZx (2020)

Los ataques contra el protocolo bZx, producidos en febrero de 2020, constituyen uno de los primeros ejemplos documentados de explotación combinada de *flash loans* y manipulación de oráculos de precio a escala en el ecosistema DeFi. Su relevancia radica no solo en las pérdidas económicas, sino en haber demostrado que la composabilidad de los protocolos DeFi puede convertirse en un vector de ataque cuando los componentes individuales no contemplan escenarios de manipulación externa [41].

En el primero de los dos incidentes, acaecido el 14 de febrero de 2020, el atacante obtuvo un préstamo flash de 10.000 ETH a través del protocolo dYdX sin necesidad de aportar colateral alguno. Con una parte de estos fondos abrió simultáneamente una posición corta apalancada sobre el par WBTC/ETH en bZx y utilizó el volumen restante para adquirir WBTC en Uniswap, manipulando artificialmente el precio de mercado del par en ese pool. El protocolo bZx, al depender de Uniswap como fuente de precios en tiempo real, calculó incorrectamente la valoración de la posición del atacante, quien la cerró obteniendo un beneficio aproximado de 350.000 dólares una vez devuelto el préstamo flash, todo dentro de una única transacción [42].

El segundo ataque, cuatro días después, siguió una mecánica similar pero orientada a la manipulación del precio del token sUSD en Kyber Network. Al inflar artificialmente su cotización, el atacante logró depositar colateral sobrevalorado en bZx y extraer en préstamo un volumen de activos muy superior al que le habría correspondido según condiciones de mercado normales [42].

Ambos ataques pusieron de manifiesto que la seguridad de un protocolo DeFi no puede analizarse de forma aislada: la dependencia de precios obtenidos de fuentes únicas y manipulables en tiempo real constituye una vulnerabilidad estructural. La adopción de oráculos descentralizados con agregación de múltiples fuentes de precio (como los proporcionados por protocolos del tipo Chainlink) y el uso de precios promediados en el tiempo (TWAP, *Time-Weighted Average Price*) son las principales contramedidas frente a este patrón de ataque.

2.5.4. Error lógico de negocio: Euler Finance (2023)

El ataque a Euler Finance, ejecutado el 13 de marzo de 2023, representa un ejemplo de elevada complejidad técnica dentro de la categoría de errores lógicos, precisamente porque

la vulnerabilidad no residía en un patrón de codificación inseguro clásico, sino en una interacción no prevista entre mecanismos financieros del propio protocolo [43].

Euler Finance era un protocolo de préstamos descentralizado en Ethereum que introducía el concepto de *sub-cuentas* para gestionar posiciones de colateral y deuda de forma independiente. La vulnerabilidad se encontraba en la función `donateToReserves()`, incorporada en una actualización anterior. Esta función permitía a un usuario transferir sus propios activos a las reservas del protocolo, pero no verificaba que dicha operación no dejase la cuenta del donante en un estado de insolvencia. En condiciones normales, esta situación habría sido detectada por el sistema de liquidación, pero el atacante aprovechó el mecanismo de autoliquidación del protocolo de una forma no contemplada en su diseño [44].

El flujo del ataque comenzó con la obtención de un préstamo flash de 30 millones de DAI desde el protocolo Aave. Con estos fondos, el atacante depositó colateral en Euler y, mediante operaciones de apalancamiento repetidas, construyó posiciones de aproximadamente 195 millones de eDAI (depósitos) y 200 millones de dDAI (deuda). A continuación, utilizó `donateToReserves()` para crear intencionalmente una posición insolvente y acto seguido activó la autoliquidación. El sistema de Euler, al calcular el bonus de liquidación sobre una posición de ese tamaño, transfirió al atacante los fondos de las reservas del protocolo en una cuantía muy superior a la deuda original [45].

La pérdida total ascendió a aproximadamente 197 millones de dólares. Sin embargo, el atacante devolvió la práctica totalidad de los fondos semanas después, tras una negociación directa con el equipo de Euler mediante mensajes publicados en la cadena de bloques. El incidente ilustra cómo la ausencia de validaciones sobre invariantes de negocio (en este caso, que ninguna operación debería permitir dejar una cuenta en estado de insolvencia) puede generar vulnerabilidades que escapan tanto a la revisión manual del código como a las herramientas de análisis estático-convencionales [44].

2.1.5. 2.6. Herramientas de Análisis

2.6.1. Slither

2.6.1.1. Descripción y características principales **Slither** es un framework de análisis estático de código abierto diseñado específicamente para la evaluación de seguridad de contratos inteligentes desarrollados en **Solidity** y **Vyper**. En la actualidad, se

posiciona como una de las herramientas más robustas y ampliamente adoptadas tanto en el ámbito académico como en la industria de la auditoría de protocolos descentralizados.

Desarrollado por la firma de seguridad Trail of Bits, su presentación formal tuvo lugar en 2019 a través del artículo de investigación Slither: A Static Analysis Framework For Smart Contracts [18], cuyos autores son Josselin Feist, Gustavo Grieco y Alex Groce

Arquitectura y Funcionamiento

El núcleo de Slither está implementado en **Python** y su principal ventaja competitiva reside en su flujo de análisis. A diferencia de otras herramientas que operan directamente sobre el bytecode de la Ethereum Virtual Machine (EVM), Slither recupera el Árbol de Sintaxis Abstracta (AST) de los contratos y lo traduce a una representación intermedia propia denominada **SlithIR**.

Esta representación, basada en *Static Single Assignment* (SSA), permite realizar análisis de flujo de datos y de control con una precisión elevada, facilitando la detección de vulnerabilidades complejas que los *linters* convencionales suelen omitir.

2.6.1.2. Capacidades y Características Principales De acuerdo con sus especificaciones técnicas y su repositorio oficial [19], Slither ofrece un conjunto de funcionalidades críticas para el desarrollo seguro:

- **Detección de Vulnerabilidades:** Capacidad para identificar código vulnerable con una tasa reducida de falsos positivos, documentada en una extensa lista de "trofeos" (vulnerabilidades reales detectadas en protocolos principales).
- **Trazabilidad:** Localiza con exactitud el punto del código fuente donde se produce la condición de error, facilitando la remediación inmediata.
- **Integración en el Ciclo de Vida del Software (DevSecOps):** Se integra de forma nativa en entornos de desarrollo como Hardhat y Foundry, además de permitir su implementación en flujos de Integración Continua (CI) y escaneo de código en GitHub.
- **Herramientas de Visualización (Printers):** Incluye funciones integradas para generar informes rápidos sobre la información crucial del contrato (jerarquía de herencia, permisos, etc.).
- **Extensibilidad:** Dispone de una API de detección que permite a los investigadores programar análisis personalizados y detectores específicos en Python.

■ **Rendimiento y Compatibilidad:**

- Soporte para contratos en Solidity desde la versión 0.4 en adelante.
- Capacidad de procesamiento del 99.9 % del código público de Solidity.
- Tiempo de ejecución promedio inferior a 1 segundo por contrato, lo que lo hace ideal para despliegues a gran escala.

2.6.1.3. Impacto en la Seguridad de Contratos Inteligentes La importancia de Slither radica en su equilibrio entre **velocidad y precisión**. Al operar sobre SlithIR, la herramienta puede realizar análisis semánticos profundos sin la carga computacional que requieren los motores de ejecución simbólica, permitiendo una validación constante durante la fase de desarrollo del contrato.

2.6.1.4. Limitaciones de Slither A pesar de su eficiencia, Slither presenta limitaciones propias del análisis estático que deben ser consideradas. La herramienta no modela interacciones complejas entre múltiples contratos de forma dinámica, lo que le impide detectar vulnerabilidades de lógica económica, como la manipulación de oráculos o ataques de *flash loans*. Asimismo, en entornos de código con alta complejidad, múltiples niveles de herencia o uso de ensamblador (*inline assembly*), Slither suele generar una tasa elevada de falsos positivos. Esto requiere que el auditor realice una validación manual exhaustiva para distinguir las vulnerabilidades reales de las falsas alarmas detectadas por el software.

2.6.1.5. Empresas y organizaciones que lo utilizan En el sector de la seguridad blockchain, Slither se ha consolidado como una herramienta de referencia. Es utilizada de manera sistemática por firmas de auditoría líderes como **ConsenSys Diligence**, **Sigma Prime** y **OpenZeppelin** para realizar el triaje inicial de los contratos. Además, su integración en flujos de trabajo de integración continua (CI/CD) es un estándar en los protocolos DeFi más relevantes, entre los que destacan **Aave**, **Uniswap**, **Yearn Finance** y **Compound**. Estas organizaciones emplean el *framework* para automatizar el escaneo de seguridad en cada actualización de sus repositorios antes del despliegue definitivo en la red.[20], [21], [22]

2.6.2. Mythril — Ejecución Simbólica (ConsenSys)

2.6.2.1. Descripción y características principales Mythril es una herramienta de seguridad de código abierto diseñada para el análisis profundo de contratos inteligentes que se ejecutan en la Ethereum Virtual Machine (EVM). A diferencia de los analizadores estáticos convencionales, Mythril se categoriza como un motor de ejecución simbólica, capaz de evaluar la seguridad de los contratos tanto a nivel de código fuente (Solidity) como directamente sobre el bytecode de la EVM [23]. Su propósito principal es identificar estados del contrato que podrían conducir a vulnerabilidades críticas mediante la exploración de rutas de ejecución que no son evidentes mediante la simple lectura del código.[24]

2.6.2.2. Arquitectura y Funcionamiento El núcleo de Mythril se basa en la **ejecución simbólica con resolución SMT (*Satisfiability Modulo Theories*)**. Su funcionamiento se puede desglosar en tres fases técnicas:

- **Desensamblado y Grafo de Flujo de Control (CFG):** Mythril descompone el *bytecode* en instrucciones de la EVM y construye un grafo que representa todos los caminos posibles que puede tomar una transacción.
- **Exploración de Estados Simbólicos:** En lugar de usar valores fijos (ej. enviar 1 ETH), utiliza variables simbólicas (x). La herramienta .ejecuta.el contrato de forma virtual, acumulando restricciones matemáticas sobre estas variables a medida que atraviesa bifurcaciones condicionales (if, require).
- **Resolución mediante Z3:** Para determinar si un estado vulnerable es alcanzable, Mythril consulta a Z3, un potente probador de teoremas desarrollado por Microsoft Research. Si el *solver* encuentra un conjunto de valores que satisfacen las condiciones para un ataque (por ejemplo, que el saldo sea cero y el llamante no sea el propietario), Mythril confirma la existencia de la vulnerabilidad.

2.6.2.3. Capacidades y Características Principales En cuanto a su potencial técnico, Mythril sobresale por su capacidad para identificar fallos lógicos de alta complejidad que están estrechamente vinculados a la manipulación del estado de la cadena de bloques.

Entre sus funciones más destacadas se encuentra la detección especializada de vulnerabilidades críticas, tales como la reentrancia clásica, el uso indebido de la instrucción DELEGATECALL, las dependencias de *timestamp* y los desbordamientos de enteros (*integer*

overflows/underflows), estos últimos especialmente relevantes en contratos desarrollados con versiones de Solidity anteriores a la 0.8.

Asimismo, la herramienta ofrece una versatilidad notable mediante su funcionalidad de análisis de Mainnet, la cual permite recuperar el bytecode directamente desde un nodo de Ethereum; esto facilita la auditoría de contratos que ya han sido desplegados incluso si el código fuente original no está disponible.

Finalmente, una de sus características más valoradas por los auditores es la generación detallada de trazas de ejecución, ya que el sistema no se limita a reportar la existencia de un error, sino que reconstruye la secuencia exacta de transacciones necesaria para recrear el exploit y validar la vulnerabilidad.

2.6.2.4. Limitaciones Técnicas A pesar de su robustez, Mythril enfrenta desafíos significativos derivados de la naturaleza de la ejecución simbólica. El obstáculo más crítico es la denominada explosión de caminos (*path explosion*), un fenómeno que ocurre en contratos con estructuras de control muy ramificadas o bucles complejos. En estos escenarios, el número de estados posibles aumenta de forma exponencial, lo que deriva frecuentemente en un consumo excesivo de memoria o en la interrupción del proceso por tiempo de espera (*timeout*) antes de completar el análisis.[25]

A esta limitación estructural se suma una tasa considerable de falsos positivos, la cual se estima en estudios independientes entre el 45 % y el 52 %. Esta imprecisión suele deberse a que la herramienta, para mantener la viabilidad del cálculo, simplifica ciertos aspectos del entorno de la blockchain, como el estado de contratos externos o variables ambientales complejas, lo que puede llevar a reportar vulnerabilidades que no son explotables en un entorno real.[26]

Finalmente, el rendimiento operativo de Mythril es notablemente inferior al de los analizadores estáticos como Slither. Debido a la carga computacional que requiere la resolución de fórmulas matemáticas mediante SMT, los tiempos de ejecución oscilan entre los 5 y 300 segundos por contrato. Esta demora depende directamente de la profundidad de exploración configurada, lo que lo hace menos ágil para procesos de integración continua que requieren una respuesta inmediata.

2.6.2.5. Ecosistema y Adopción En la actualidad, Mythril se consolida como un pilar fundamental dentro de la industria de la seguridad blockchain. Su relevancia técnica y

trayectoria lo han posicionado como el motor central de **MythX**, la plataforma de análisis de seguridad profesional bajo modelo SaaS desarrollada por ConsenSys. Esta integración permite a los desarrolladores acceder a las capacidades de Mythril directamente desde sus entornos de trabajo habituales.

Asimismo, la herramienta se ha convertido en un estándar dentro de los flujos de trabajo de **auditoría profesional**. Firmas de prestigio internacional, tales como **ConsenSys Diligence** y **Halborn**, emplean Mythril de forma sistemática para complementar sus revisiones manuales, utilizando su capacidad de exploración de estados para detectar errores lógicos que podrían pasar desapercibidos en una lectura convencional del código.

Más allá del sector comercial, Mythril posee una fuerte presencia en la **comunidad académica**. Su naturaleza de código abierto y su arquitectura basada en el *solver* Z3 lo convierten en una base tecnológica recurrente para investigaciones avanzadas. Es utilizado frecuentemente en estudios sobre verificación formal y en el desarrollo de nuevos protocolos de seguridad destinados a proteger el ecosistema de las finanzas descentralizadas (DeFi) [24], [25].

2.6.3. Echidna — Fuzzing Basado en Propiedades (Trail of Bits)

2.6.3.1. Descripción y características principales Echidna es una herramienta de código abierto desarrollada por Trail of Bits, diseñada específicamente para el análisis de seguridad de contratos inteligentes en Ethereum mediante la técnica de *fuzzing* basado en propiedades. A diferencia de las pruebas unitarias tradicionales, que se limitan a verificar resultados ante entradas específicas, este marco de trabajo busca falsar invariantes o predicados definidos por el usuario, permitiendo identificar fallos lógicos complejos que suelen pasar desapercibidos en procesos de testeo convencionales. Implementado en Haskell, el programa destaca por su facilidad de uso, ya que no requiere configuraciones complejas ni el despliegue previo de los contratos en una red local.[27]

2.6.3.2. Arquitectura y Funcionamiento El funcionamiento de Echidna se fundamenta en la generación de campañas de *fuzzing* basadas en gramática, utilizando el ABI (*Application Binary Interface*) del contrato para interactuar con él. El flujo de trabajo comienza cuando el desarrollador define propiedades de seguridad en el código Solidity, normalmente funciones con el prefijo `echidna_` que deben devolver siempre un valor booleano verdadero. A partir de estas definiciones, la herramienta ejecuta millones de secuencias de

transacciones aleatorias y sofisticadas con el objetivo de alcanzar un estado del contrato que rompa dichas reglas.[28]

2.6.3.3. Capacidades y Características Principales Una de las capacidades más potentes de este fuzzer es su capacidad de reducción de casos de prueba. Cuando Echidna detecta una violación de una propiedad o una aserción de Solidity, el sistema realiza una simplificación automática para reportar la secuencia mínima de transacciones necesaria para reproducir el error. Esta eficiencia en la detección y reporte ha quedado demostrada en entornos reales de producción; ya en la fecha de publicación de su *paper* original, la herramienta había sido validada con éxito en más de diez auditorías de seguridad de gran escala, consolidándose como un estándar en el ecosistema de seguridad de *smart contracts*. [27]

2.6.3.4. Limitaciones Técnicas A pesar de su eficacia, Echidna presenta restricciones técnicas inherentes a su metodología de prueba. En primer lugar, la calidad del análisis depende de la calidad de las propiedades definidas por el usuario; si el desarrollador no define correctamente los invariantes, Echidna no podrá identificar fallos de lógica específicos [29].

Por otro lado, la herramienta puede enfrentar problemas de rendimiento en contratos con funciones extremadamente costosas en términos de gas o con estructuras de datos ineficientes, lo que ralentiza la generación de transacciones [28].

Asimismo, aunque es excelente para detectar errores lógicos, tiene dificultades para analizar contratos que dependen de condiciones externas muy específicas de la red que no pueden ser fácilmente simuladas en su entorno local de pruebas [29].

2.6.3.5. Ecosistema y Adopción

Debido a su eficacia demostrada, Echidna se ha convertido en una herramienta estándar en la industria:

- **Trail of Bits:** Como creadores de la herramienta, la utilizan en todas sus auditorías de alto perfil para protocolos DeFi [28].
- **Audidores Profesionales:** Firmas de ciberseguridad blockchain emplean Echidna para validar la robustez de protocolos antes de su despliegue en la red principal [29].
- **Equipos de Desarrollo:** Proyectos de infraestructura y finanzas descentralizadas

integran Echidna en sus tuberías de **Integración Continua (CI/CD)** para garantizar que cada actualización de código mantenga los invariantes de seguridad [27].

2.1.6. 2.7. Síntesis y limitaciones del estado del arte

El análisis realizado muestra que el ecosistema de contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM) concentra actualmente la mayor parte de herramientas, investigaciones y estándares relacionados con la seguridad en blockchain. Solidity se ha consolidado como el lenguaje dominante en el desarrollo de aplicaciones descentralizadas y protocolos DeFi, lo que ha favorecido la aparición de múltiples herramientas especializadas para el análisis de vulnerabilidades.

No obstante, el estado del arte también evidencia limitaciones relevantes. Herramientas como Slither, Mythril o Echidna utilizan enfoques distintos y generan resultados heterogéneos, dificultando la correlación de hallazgos y aumentando la necesidad de validación manual por parte de auditores especializados. Asimismo, muchas soluciones presentan dificultades para priorizar riesgos o contextualizar el impacto real de las vulnerabilidades detectadas.

Por otro lado, gran parte de las vulnerabilidades analizadas dependen principalmente del modelo de ejecución de la EVM y del diseño de los contratos inteligentes, siendo extrapolables a distintas redes compatibles como Ethereum, BNB Smart Chain, Polygon o Arbitrum. Por este motivo, el presente trabajo utilizará BNB Smart Chain Testnet como entorno de pruebas, aprovechando su compatibilidad con Solidity y la reducción de costes durante el desarrollo experimental.

En conjunto, estas limitaciones justifican la necesidad de desarrollar soluciones que permitan integrar múltiples herramientas de análisis, normalizar resultados y facilitar una interpretación más estructurada de los hallazgos de seguridad en contratos inteligentes.

3. Objetivos concretos y metodología de trabajo

3.1. 3. Objetivos concretos y metodología de trabajo

3.1.1. 3.1. Objetivo general

El objetivo general del presente Trabajo Fin de Máster consiste en diseñar, implementar y evaluar una librería en Python orientada al análisis de seguridad de contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM), capaz de integrar y correlacionar resultados procedentes de distintas herramientas de auditoría.

La propuesta busca facilitar el análisis de vulnerabilidades en contratos desarrollados en Solidity mediante un enfoque modular que permita unificar resultados heterogéneos y mejorar la interpretación de los hallazgos detectados.

3.1.2. 3.2. Objetivos específicos

Para alcanzar el objetivo general se plantean los siguientes objetivos específicos:

- Analizar las principales vulnerabilidades de seguridad presentes en contratos inteligentes desarrollados en Solidity y las técnicas utilizadas para su detección.
- Estudiar el funcionamiento, capacidades y limitaciones de herramientas de análisis como Slither, Mythril y Echidna.
- Diseñar una arquitectura modular en Python que permita integrar distintas herramientas de análisis de contratos inteligentes.
- Implementar mecanismos de normalización y correlación de resultados para unificar hallazgos procedentes de diferentes herramientas.
- Desarrollar un sistema de generación de informes que facilite la interpretación de vulnerabilidades detectadas durante el análisis.
- Evaluar el funcionamiento de la librería utilizando contratos vulnerables conocidos y contratos reales de código abierto compatibles con la EVM.

3.1.3. 3.3. Metodología del trabajo

Con el fin de alcanzar los objetivos planteados, el desarrollo del trabajo se estructuró en varias fases que combinaron tanto el análisis teórico del problema como la implementación práctica de la solución propuesta.

En una primera fase se realizó un estudio del estado del arte relacionado con blockchain, contratos inteligentes, vulnerabilidades en Solidity y técnicas de auditoría de seguridad. Asimismo, se analizaron distintas herramientas existentes para identificar sus capacidades, limitaciones y posibilidades de integración.

Posteriormente, se definió la arquitectura de la librería, estableciendo los módulos necesarios para la ejecución de herramientas externas, el procesamiento de resultados y la generación de informes. Durante esta etapa se priorizó un diseño modular y extensible que facilitase futuras ampliaciones y la incorporación de nuevos mecanismos de análisis.

A continuación, se llevó a cabo la implementación de la solución utilizando Python como lenguaje principal. La librería desarrollada integró herramientas externas de análisis y permitió automatizar el procesamiento, normalización y correlación de los resultados obtenidos.

Finalmente, se realizó una evaluación experimental utilizando contratos inteligentes vulnerables y casos reales obtenidos de repositorios públicos. Los resultados obtenidos permitieron analizar el comportamiento de la solución propuesta y valorar su utilidad como apoyo al análisis de seguridad de contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM).

4. Desarrollo específico de la contribución

4.1. 4. Desarrollo de la solución propuesta

Una vez estudiadas las principales vulnerabilidades presentes en los contratos inteligentes y analizadas las herramientas existentes para su detección, en este capítulo se presenta la solución desarrollada para dar respuesta a los objetivos planteados en el presente Trabajo Fin de Máster.

La propuesta realizada, denominada **EVMAudit**, consiste en una librería desarrollada en Python orientada al análisis de seguridad de contratos inteligentes compatibles con la Ethereum Virtual Machine (EVM). La herramienta integra diferentes técnicas de análisis mediante la ejecución combinada de Slither, Mythril y Echidna, con el objetivo de aprovechar las capacidades específicas de cada una de ellas y proporcionar una visión más completa de las vulnerabilidades detectadas.

Con el fin de favorecer la mantenibilidad y la extensibilidad del sistema, se adoptó una arquitectura modular en la que cada componente implementa una responsabilidad específica dentro del proceso de análisis. De este modo, la solución permite automatizar la ejecución de las herramientas externas, procesar y correlacionar los resultados obtenidos y generar información adicional que facilite la validación posterior de las vulnerabilidades identificadas.

A lo largo de este capítulo se describen los requisitos que guiaron el desarrollo de la herramienta, la arquitectura diseñada y los distintos módulos implementados. Finalmente, se presenta la evaluación experimental realizada con el objetivo de analizar el comportamiento de la solución propuesta y valorar su utilidad como apoyo al proceso de auditoría de contratos inteligentes.

!TODO: referencia cruzada a la Sección 3.1 (objetivo general).

!TODO: referencia cruzada a la Sección 3.2 (objetivos específicos).

!TODO: referencia cruzada a la Sección 3.3 (metodología).

4.2. 4.1.Requisitos

4.2.1. 4.1. Identificación de requisitos

A partir del análisis realizado durante el estudio del estado del arte y de las limitaciones observadas en las herramientas existentes, se definieron una serie de requisitos que sirvieron como base para el diseño e implementación de EVMAudit.

Los requisitos establecidos se clasifican en requisitos funcionales, requisitos no funcionales y requisitos de seguridad.

4.1.1. Requisitos funcionales

Los requisitos funcionales describen las capacidades que debe proporcionar la herramienta para satisfacer los objetivos del trabajo.

$\text{RF-ID} = 0.0350 \cdot \text{RF-ID} + 0.9650 \cdot \text{RF-ID}$

Descripción

RF-01 La herramienta debe permitir analizar contratos inteligentes desarrollados en Solidity.

RF-02 La herramienta debe detectar automáticamente el nombre del contrato cuando no se especifique manualmente.

RF-03 La herramienta debe configurar automáticamente la versión del compilador Solidity necesaria para el análisis.

RF-04 La herramienta debe ejecutar Slither como herramienta de análisis estático.

RF-05 La herramienta debe ejecutar Mythril como herramienta de análisis mediante ejecución simbólica.

RF-06 La herramienta debe almacenar las salidas originales de las herramientas utilizadas.

RF-07 La herramienta debe normalizar los resultados generados por Slither y Mythril a una estructura común.

RF-08 La herramienta debe asociar los hallazgos detectados con identificadores SWC.

RF-09 La herramienta debe correlacionar hallazgos equivalentes procedentes de distintas herramientas.

RF-10 La herramienta debe indicar el nivel de confianza de los hallazgos en función de las herramientas que hayan detectado cada vulnerabilidad.

RF-11 La herramienta debe generar automáticamente propiedades para su validación mediante Echidna.

RF-12 La herramienta debe ejecutar Echidna sobre los wrappers generados.

RF-13 La herramienta debe generar un informe final consolidado.

RF-14 La herramienta debe conservar la trazabilidad de los resultados originales.

4.1.2. Requisitos no funcionales

Los requisitos no funcionales definen las características de calidad que debe presentar la solución desarrollada.

⌈@ @p() * 0.0600 @p() * 0.9400@ ID

Descripción

RNF-01 La arquitectura de la herramienta debe ser modular.

RNF-02 La solución debe permitir la incorporación de nuevas herramientas en futuras versiones.

RNF-03 Los resultados intermedios deben almacenarse para garantizar la reproducibilidad del análisis.

RNF-04 La solución debe presentar un bajo acoplamiento entre módulos.

RNF-05 La herramienta debe facilitar el mantenimiento y evolución futura del sistema.

RNF-06 La lógica de análisis debe permanecer desacoplada de las interfaces de usuario.

4.1.3. Requisitos de seguridad

Debido a la naturaleza del trabajo, se definieron además una serie de requisitos orientados a mejorar la robustez del proceso de análisis.

⌈@ @p() * 0.0505 @p() * 0.9495@ ID

Descripción

RS-01 La herramienta debe detectar la ausencia de dependencias externas necesarias para el análisis.

RS-02 La herramienta debe gestionar errores producidos por herramientas externas.

RS-03 La herramienta debe controlar tiempos máximos de ejecución.

RS-04 La herramienta debe preservar las evidencias originales obtenidas durante el análisis.

RS-05 La herramienta debe permitir reproducir los resultados obtenidos.

!TODO: insertar tabla definitiva con formato Word.

!TODO: referencia cruzada a las herramientas descritas en la Sección 2.X.

!TODO: referencia cruzada a las vulnerabilidades descritas en la Sección 2.X.

4.3. 4.2.Arquitectura

4.3.1. 4.2. Arquitectura general de EVMAudit

Con el objetivo de satisfacer los requisitos definidos anteriormente, se diseñó una arquitectura modular que permitiese separar las distintas fases del proceso de análisis y facilitar la incorporación de nuevas funcionalidades en futuras versiones.

La arquitectura implementada divide el sistema en varios módulos especializados encargados de:

- ejecutar las herramientas externas
- procesar y normalizar los resultados obtenidos
- correlacionar vulnerabilidades equivalentes
- generar propiedades para Echidna
- ejecutar pruebas de fuzzing
- construir el informe final.

Esta separación de responsabilidades permite reducir el acoplamiento entre componentes y facilita la mantenibilidad del sistema.

4.2.1. Pipeline de análisis

El flujo general seguido por EVMAudit comienza con la recepción del contrato inteligente que se desea analizar. Posteriormente, la herramienta ejecuta Slither y Mythril para obtener resultados procedentes de análisis estático y ejecución simbólica.

Una vez finalizada esta fase, los resultados generados son normalizados y transformados a una estructura común. Posteriormente, se aplica el mecanismo de correlación implemen-

tado por EVMAudit con el objetivo de unificar vulnerabilidades equivalentes y aumentar la confianza de los hallazgos obtenidos.

A partir de los resultados correlacionados, se generan automáticamente propiedades para Echidna, construyendo un wrapper específico que permite realizar una fase adicional de validación mediante fuzzing.

Finalmente, la herramienta genera un informe consolidado que agrupa toda la información obtenida durante el proceso.

!TODO: insertar diagrama del pipeline completo.

!TODO: insertar referencia a la Figura X.

4.2.2. Estructura modular

La implementación de EVMAudit se organiza en varios módulos independientes, cada uno de ellos responsable de una fase concreta del análisis.

Los principales componentes que forman la solución son:

- Runner.
- Normalizer.
- Correlator.
- SWC Catalog.
- Echidna Adapter.
- Reporter.
- Exceptions.

Esta organización permite modificar o ampliar cada componente de forma independiente, favoreciendo la reutilización y evolución futura de la herramienta.

!TODO: insertar diagrama UML de paquetes.

!TODO: revisar nombres definitivos de los módulos.

4.2.3. Separación entre lógica de análisis e interfaces

Uno de los principios de diseño adoptados durante el desarrollo fue desacoplar completamente la lógica de análisis de las interfaces de usuario.

Por este motivo, EVMAudit fue implementada como una librería independiente, permitiendo que la funcionalidad principal pueda ser reutilizada desde distintos entornos sin necesidad de modificar el código interno.

Esta decisión permitió desarrollar posteriormente una aplicación web que utiliza la librería como motor de análisis, demostrando la reutilización de la solución propuesta.

Los detalles relacionados con la aplicación web desarrollada y los mecanismos de distribución utilizados se describen en los anexos del documento.

!TODO: referencia cruzada al apartado 4.9.

!TODO: referencia cruzada al Anexo A.

!TODO: referencia cruzada al Anexo B.

!TODO: referencia cruzada al Anexo C.

4.4. 4.3.Runner

4.4.1. 4.3. Módulo Runner

El módulo Runner constituye la capa encargada de la interacción con las herramientas externas utilizadas durante el proceso de análisis. Su principal responsabilidad consiste en gestionar la ejecución de Slither, Mythril y Echidna, así como preparar el entorno necesario para garantizar la correcta realización del análisis.

La existencia de un módulo específico para esta tarea permite desacoplar la lógica propia de EVMAudit de las particularidades de cada herramienta, facilitando la incorporación de nuevas soluciones de análisis en futuras versiones.

!TODO: insertar referencia a la Figura X (diagrama de arquitectura).

4.3.1. Responsabilidades del módulo

Las principales responsabilidades implementadas por el módulo Runner son las siguientes:

- detección automática del nombre del contrato
- configuración de la versión del compilador Solidity
- ejecución de Slither
- ejecución de Mythril

- ejecución de Echidna
- almacenamiento de resultados intermedios
- gestión de errores producidos durante la ejecución.

Gracias a esta aproximación, el resto de módulos del sistema pueden trabajar de forma independiente sin necesidad de conocer los detalles específicos de cada herramienta externa.

4.3.2. Detección automática del nombre del contrato

Durante las primeras fases de desarrollo se observó que el nombre del fichero Solidity no siempre coincide con el nombre del contrato definido en su interior.

Por este motivo se implementó un mecanismo de detección automática que analiza el código fuente y extrae el nombre real del contrato utilizando expresiones regulares.

Esta decisión permite evitar errores durante las fases posteriores del análisis y elimina la necesidad de que el usuario proporcione manualmente dicha información.

Además, este mecanismo resulta especialmente útil cuando la librería es utilizada desde otras aplicaciones externas, ya que reduce la cantidad de parámetros que deben proporcionarse.

!TODO: insertar fragmento de código de `detect_contract_name()`.

4.3.3. Configuración automática del compilador

Las herramientas de análisis empleadas dependen de la versión del compilador Solidity utilizada por el contrato.

Con el objetivo de garantizar la reproducibilidad del análisis y evitar incompatibilidades entre versiones, EVMAudit analiza automáticamente la directiva `pragma solidity` presente en el contrato y configura la versión correspondiente mediante la utilidad `solc-select`.

Este mecanismo permite adaptar dinámicamente el entorno de ejecución y facilita el análisis de contratos desarrollados con diferentes versiones del lenguaje.

!TODO: insertar referencia cruzada a la Sección 2.X (Solidity y compilador).

!TODO: insertar fragmento de código de `_set_solc_version()`.

4.3.4. Ejecución de Slither

Slither constituye la primera herramienta utilizada durante el proceso de análisis debido a su rapidez y a la gran cantidad de detectores disponibles.

La función encargada de su ejecución verifica previamente que la herramienta se encuentre instalada y posteriormente lanza el análisis sobre el contrato proporcionado.

Una vez finalizada la ejecución, la salida obtenida se almacena en formato JSON para garantizar la trazabilidad de los resultados.

Durante la implementación fue necesario contemplar ciertos comportamientos específicos de Slither. En particular, la herramienta devuelve el código de salida 255 cuando detecta vulnerabilidades, aunque dicho comportamiento no representa un error real.

Por este motivo, EVMAudit incorpora una lógica específica para interpretar correctamente este caso y continuar con el proceso de análisis.

!TODO: insertar fragmento de código de `run_slither()`.

!TODO: referencia cruzada a la Sección 2.X (Slither).

4.3.5. Ejecución de Mythril

La segunda fase del análisis se realiza mediante Mythril, herramienta basada en ejecución simbólica.

A diferencia de Slither, Mythril presenta tiempos de ejecución superiores y requiere parámetros adicionales relacionados con la profundidad máxima de exploración y el tiempo máximo permitido para completar el análisis.

Durante el desarrollo se incorporaron mecanismos para:

- limitar el tiempo máximo de ejecución
- controlar posibles errores durante el análisis
- procesar las salidas generadas por la herramienta
- preservar los resultados originales.

Estas medidas permiten evitar que un contrato especialmente complejo bloquee el proceso completo de análisis.

!TODO: insertar fragmento de código de `run_mythril()`.

!TODO: referencia cruzada a la Sección 2.X (Mythril).

4.3.6. Ejecución de Echidna

Una vez generadas las propiedades correspondientes, EVMAudit ejecuta Echidna para realizar una fase adicional de validación mediante fuzzing.

La integración de Echidna presentó una complejidad superior a la observada en Slither y Mythril debido a ciertas particularidades de la herramienta y a las diferencias existentes entre versiones.

Durante el desarrollo fue necesario incorporar mecanismos específicos para:

- reconstruir nombres de propiedades
- interpretar adecuadamente las salidas generadas
- distinguir entre pruebas superadas y vulnerabilidades explotables
- gestionar posibles inconsistencias en los resultados.

Estas adaptaciones permitieron integrar Echidna dentro del flujo general de EVMAudit sin alterar el resto de componentes de la arquitectura.

!TODO: insertar fragmento de código de `run_echidna()`.

!TODO: referencia cruzada a la Sección 2.X (Echidna).

4.3.7. Persistencia de resultados intermedios

Con el fin de favorecer la trazabilidad y la reproducibilidad del análisis, EVMAudit almacena las salidas originales generadas por las distintas herramientas utilizadas.

Esta decisión permite:

- inspeccionar manualmente los resultados obtenidos
- reproducir análisis posteriores
- depurar posibles errores
- conservar evidencias originales.

La preservación de las salidas originales resulta especialmente relevante en el contexto de auditorías de seguridad, donde la trazabilidad de los hallazgos constituye un aspecto fundamental.

4.5. 4.4.Normalizer

4.5.1. 4.4. Módulo de normalización

Uno de los principales problemas identificados durante el estudio de las herramientas existentes fue la heterogeneidad de los formatos utilizados para representar las vulnerabilidades detectadas.

Slither y Mythril generan estructuras JSON completamente diferentes, tanto en la forma de representar la severidad como en la descripción de los hallazgos o la identificación de las localizaciones afectadas.

Esta situación dificulta la comparación directa de resultados y hace necesaria una fase previa de transformación antes de poder aplicar mecanismos de correlación.

Para resolver este problema se desarrolló un módulo de normalización encargado de transformar las salidas heterogéneas en una estructura común.

4.4.1. Objetivos del proceso de normalización

El proceso de normalización persigue varios objetivos:

- unificar la representación de vulnerabilidades;
- facilitar la correlación entre herramientas;
- preservar las evidencias originales;
- simplificar la generación posterior de informes;
- desacoplar el resto de módulos de las particularidades de cada herramienta.

Gracias a ello, los componentes posteriores pueden operar sobre una estructura homogénea independientemente del origen de los resultados.

4.4.2. Estructura común de los hallazgos

Cada vulnerabilidad normalizada se representa mediante un conjunto común de atributos.

Entre los campos principales se encuentran:

- título;

- descripción;
- severidad;
- categoría;
- contrato afectado;
- función afectada;
- localización;
- identificador SWC;
- herramienta de origen;
- evidencia original.

La inclusión de la salida original permite mantener la trazabilidad del análisis y facilita la revisión manual de los resultados.

!TODO: insertar diagrama UML del objeto Finding.

!TODO: revisar nombres exactos de los atributos.

4.4.3. Normalización de resultados de Slither

La función de normalización correspondiente a Slither transforma la estructura JSON generada por la herramienta y extrae la información relevante necesaria para construir los hallazgos comunes utilizados por EVMAudit.

Durante esta fase se realiza además la asociación de los detectores propios de Slither con los identificadores SWC correspondientes.

Este proceso permite traducir la terminología específica de Slither a una representación independiente de la herramienta utilizada.

!TODO: insertar fragmento de código de `normalize_slither_output()`.

4.4.4. Normalización de resultados de Mythril

De forma análoga, la salida generada por Mythril es procesada para obtener una representación equivalente a la utilizada por Slither.

La transformación permite unificar:

- niveles de severidad;

- nombres de vulnerabilidades;
- identificadores SWC;
- información contextual del hallazgo.

Gracias a ello, ambas herramientas pueden ser tratadas posteriormente de forma transparente por el módulo de correlación.

!TODO: insertar fragmento de código de `normalize_mythril_output()`.

4.4.5. Clasificación de vulnerabilidades

Además de unificar la representación de los hallazgos, el módulo de normalización clasifica las vulnerabilidades detectadas siguiendo las categorías definidas durante el estudio del estado del arte.

Entre las categorías utilizadas se encuentran:

- vulnerabilidades de control de acceso;
- vulnerabilidades económicas;
- vulnerabilidades relacionadas con la lógica de negocio;
- vulnerabilidades asociadas a la ejecución del contrato.

Esta clasificación permite mantener la coherencia entre la parte teórica del trabajo y la implementación desarrollada.

!TODO: referencia cruzada a la Sección 2.X (clasificación de vulnerabilidades).

4.4.6. Preservación de evidencias

Uno de los principios adoptados durante el diseño de EVMAudit fue evitar la pérdida de información durante las fases intermedias del análisis.

Por este motivo, cada hallazgo normalizado conserva una referencia a la salida original generada por la herramienta correspondiente.

Esta decisión facilita:

- la revisión manual por parte del auditor;
- la trazabilidad del proceso;

- la depuración de errores;
- la reproducibilidad de los resultados.

La conservación de evidencias resulta especialmente importante en entornos de auditoría y análisis forense, donde es necesario justificar el origen de cada vulnerabilidad identificada.

!TODO: insertar diagrama del proceso de normalización.

!TODO: insertar referencia a la Figura X.

4.6. 4.5. Correlator

4.6.1. 4.5. Módulo de correlación

Una vez normalizados los resultados obtenidos por las herramientas de análisis, EVMAudit aplica un proceso de correlación cuyo objetivo consiste en identificar vulnerabilidades equivalentes detectadas por distintas herramientas y agruparlas en un único hallazgo.

Este módulo constituye el núcleo de la contribución desarrollada en el presente Trabajo Fin de Máster, ya que permite reducir la redundancia de información y proporcionar una visión más estructurada de las vulnerabilidades detectadas.

La necesidad de incorporar esta fase surge de una de las principales limitaciones observadas durante el estudio del estado del arte. Aunque herramientas como Slither y Mythril son capaces de detectar un gran número de vulnerabilidades, cada una de ellas presenta los resultados utilizando nomenclaturas y estructuras diferentes, generando además múltiples duplicidades que dificultan la revisión manual por parte del auditor.

Por este motivo, EVMAudit incorpora un mecanismo propio de correlación encargado de consolidar la información procedente de ambas herramientas.

4.5.1. Objetivos del proceso de correlación

Los objetivos perseguidos por este módulo son los siguientes:

- reducir la redundancia de información
- agrupar vulnerabilidades equivalentes
- incrementar la confianza en los hallazgos detectados

- facilitar la interpretación de resultados
- proporcionar una representación unificada de las vulnerabilidades.

La correlación no pretende eliminar completamente los falsos positivos generados por las herramientas utilizadas, sino proporcionar una capa adicional de análisis que facilite el trabajo posterior del auditor.

4.5.2. Estrategia de correlación

El proceso implementado en EVMAudit se basa en una estrategia de correlación mediante reglas.

Dos vulnerabilidades se consideran equivalentes cuando cumplen simultáneamente las siguientes condiciones:

- afectan al mismo contrato
- afectan a la misma función
- poseen el mismo identificador SWC.

De este modo, la clave utilizada para la correlación puede expresarse de la siguiente forma:

□ Contrato + Función + SWC

Esta aproximación permite agrupar vulnerabilidades que, aunque procedan de herramientas diferentes, representan realmente el mismo problema de seguridad.

La elección del identificador SWC como criterio común permite disponer de una referencia independiente de la nomenclatura utilizada por cada herramienta.

!TODO: insertar referencia cruzada a la Sección 2.X (SWC Registry).

!TODO: insertar pseudocódigo del algoritmo de correlación.

4.5.3. Hallazgos confirmados y hallazgos detectados

Una vez realizado el proceso de agrupación, EVMAudit distingue entre dos tipos de vulnerabilidades.

Hallazgos confirmados Se consideran confirmados aquellos hallazgos que han sido detectados por más de una herramienta.

La coincidencia entre distintas técnicas de análisis aumenta la confianza asociada a la vulnerabilidad y proporciona una mayor evidencia de su existencia.

Hallazgos detectados Corresponden a vulnerabilidades identificadas por una única herramienta.

Aunque presentan un menor nivel de confianza, siguen siendo incluidas en el informe final con el objetivo de evitar la pérdida de información potencialmente relevante.

Esta clasificación permite priorizar posteriormente la revisión manual realizada por el auditor.

4.5.4. Nivel de confianza

Con el objetivo de proporcionar información adicional sobre la fiabilidad de cada hallazgo, EVMAudit asigna un nivel de confianza basado en las herramientas que han detectado la vulnerabilidad.

La confianza asociada no representa una medida probabilística, sino un indicador orientativo derivado del número de herramientas que coinciden en el mismo hallazgo.

Este mecanismo permite distinguir aquellas vulnerabilidades respaldadas por varias herramientas de aquellas detectadas únicamente por una de ellas.

No obstante, la interpretación final continúa dependiendo del criterio del auditor, ya que la correlación implementada no sustituye la validación manual.

!TODO: revisar nomenclatura definitiva utilizada por el código (`confidence_score`).

4.5.5. Gestión de severidades

Cuando una vulnerabilidad es detectada por varias herramientas, puede ocurrir que cada una de ellas asigne niveles de severidad diferentes.

Para resolver esta situación, EVMAudit conserva la severidad más alta reportada por las herramientas implicadas.

Este enfoque se adopta siguiendo un criterio conservador, priorizando la revisión de aquellas vulnerabilidades que potencialmente puedan tener un mayor impacto.

4.5.6. Preservación de evidencias

Durante el proceso de correlación se mantiene la información procedente de todas las herramientas que han participado en la detección.

De esta forma, cada hallazgo correlacionado conserva:

- las herramientas que lo han identificado
- las evidencias originales asociadas
- las localizaciones afectadas
- la información contextual proporcionada por cada herramienta.

Esta decisión garantiza la trazabilidad del análisis y facilita la revisión manual posterior.

4.5.7. Beneficios del proceso de correlación

La incorporación de esta fase proporciona diversas ventajas:

- reducción de duplicidades
- mejora de la legibilidad del informe final
- aumento de la confianza en determinados hallazgos
- simplificación del proceso de auditoría
- independencia respecto a las herramientas utilizadas.

La correlación constituye, por tanto, uno de los principales elementos diferenciadores de EVMAudit frente al uso individual de las herramientas integradas.

!TODO: insertar diagrama del proceso de correlación.

!TODO: insertar ejemplo real antes y después de la correlación.

!TODO: insertar referencia a la Figura X.

4.7. 4.6.SWC Catalog

4.7.1. 4.6. Catálogo de vulnerabilidades SWC

Con el objetivo de desacoplar la fase de detección de vulnerabilidades de las fases posteriores de validación y generación de pruebas, se implementó un catálogo de vulnerabilidades basado en la clasificación Smart Contract Weakness Classification (SWC).

Este catálogo constituye una capa intermedia que permite asociar los hallazgos detectados con información adicional independiente de las herramientas utilizadas.

Gracias a ello, EVMAudit puede reutilizar la información obtenida durante el proceso de correlación y emplearla posteriormente para la generación automática de propiedades orientadas a Echidna.

4.6.1. Objetivos del catálogo

El catálogo desarrollado persigue los siguientes objetivos:

- unificar la representación de vulnerabilidades
- desacoplar la lógica de generación de pruebas
- facilitar la incorporación de nuevas plantillas
- centralizar información sobre las debilidades conocidas
- favorecer la mantenibilidad del sistema.

Esta aproximación permite que las fases posteriores del análisis no dependan directamente de las particularidades de Slither o Mythril.

4.6.2. Estructura del catálogo

Cada entrada del catálogo contiene información asociada a una vulnerabilidad concreta.

Entre los datos almacenados se encuentran:

- identificador SWC
- nombre de la vulnerabilidad
- descripción

- severidad
- plantilla asociada
- limitaciones conocidas
- posibilidad de validación automática.

La información almacenada permite enriquecer los hallazgos correlacionados y proporcionar información adicional durante las fases posteriores del análisis.

!TODO: insertar tabla con varios ejemplos del catálogo.

4.6.3. Asociación entre detectores y SWC

Una de las principales dificultades observadas durante el desarrollo fue la ausencia de una correspondencia directa entre los detectores utilizados por cada herramienta.

Para resolver este problema se definieron tablas de asociación que permiten traducir los detectores específicos a identificadores SWC.

Gracias a esta estrategia es posible utilizar un lenguaje común durante todo el proceso de análisis y evitar dependencias directas respecto a la nomenclatura propia de cada herramienta.

Esta decisión resulta especialmente relevante para el módulo de correlación descrito anteriormente.

!TODO: insertar referencia cruzada a la Sección 4.4.

4.6.4. Reutilización del catálogo

Además de servir como mecanismo de clasificación, el catálogo es utilizado posteriormente durante la generación de propiedades para Echidna.

A partir del identificador SWC asociado a cada vulnerabilidad, EVMAudit recupera automáticamente la plantilla correspondiente y genera las estructuras necesarias para la fase de fuzzing.

Esta aproximación permite separar claramente:

- detección
- correlación
- generación de pruebas.

La independencia entre estas fases facilita futuras ampliaciones y simplifica el mantenimiento del sistema.

4.6.5. Extensibilidad del catálogo

La utilización de un catálogo independiente permite incorporar nuevas vulnerabilidades sin necesidad de modificar el resto de módulos implementados.

De esta forma, la incorporación de nuevas entradas únicamente requiere:

1. definir el identificador SWC correspondiente
2. añadir la información descriptiva asociada
3. incorporar la plantilla de generación deseada.

Este diseño favorece la evolución futura de la herramienta y permite adaptar EVMAudit a nuevas versiones del ecosistema Ethereum.

!TODO: insertar diagrama de relación entre SWC Catalog y Echidna Adapter.

!TODO: insertar referencia a la Figura X.

4.8. 4.7.Echidna adaptar

4.8.1. 4.7. Adaptador para Echidna y generación automática de propiedades

Una vez finalizado el proceso de correlación, EVMAudit incorpora una fase adicional orientada a la validación dinámica de determinadas vulnerabilidades mediante fuzzing.

Para ello, se desarrolló un adaptador específico encargado de transformar los hallazgos correlacionados en propiedades compatibles con Echidna, permitiendo complementar los resultados obtenidos mediante análisis estático y ejecución simbólica.

La incorporación de esta fase responde a uno de los objetivos específicos planteados en el trabajo, consistente en combinar distintas técnicas de análisis con el fin de obtener una visión más completa del estado de seguridad del contrato analizado.

!TODO: referencia cruzada a la Sección 3.2 (objetivos específicos).

4.7.1. Objetivos del adaptador

El módulo desarrollado persigue los siguientes objetivos:

- reutilizar la información obtenida durante la fase de correlación
- generar automáticamente propiedades para Echidna
- reducir la intervención manual del auditor
- proporcionar una fase adicional de validación
- mantener desacopladas las distintas fases del análisis.

La existencia de este componente permite integrar el fuzzing dentro del pipeline general de EVMAudit sin introducir dependencias directas entre las herramientas utilizadas.

4.7.2. Generación de wrappers

Echidna requiere que las propiedades de seguridad estén implementadas como funciones Solidity dentro de un contrato específico.

Con el objetivo de automatizar este proceso, EVMAudit genera dinámicamente un contrato wrapper que hereda del contrato original e incorpora las propiedades correspondientes a las vulnerabilidades detectadas.

El proceso seguido puede resumirse en las siguientes etapas:

1. Recepción de los hallazgos correlacionados.
2. Consulta del catálogo SWC.
3. Obtención de las plantillas asociadas.
4. Sustitución de parámetros.
5. Generación del wrapper final.
6. Ejecución de Echidna.

Gracias a esta aproximación, la generación de pruebas se realiza de forma completamente automática.

!TODO: insertar diagrama del proceso de generación de wrappers.

!TODO: insertar referencia a la Figura X.

4.7.3. Plantillas de propiedades

Cada vulnerabilidad soportada dispone de una plantilla específica que describe la propiedad que deberá ser evaluada por Echidna.

Estas plantillas permiten traducir información abstracta procedente del análisis estático a estructuras ejecutables compatibles con el motor de fuzzing.

La utilización de plantillas aporta varias ventajas:

- reutilización de código
- independencia respecto a las herramientas de detección
- facilidad de mantenimiento
- incorporación sencilla de nuevas vulnerabilidades.

!TODO: insertar ejemplo de una plantilla real.

4.7.4. Vulnerabilidades testables y no testables

Durante el desarrollo se observó que no todas las vulnerabilidades pueden verificarse automáticamente mediante fuzzing.

Por este motivo, el adaptador distingue entre:

Vulnerabilidades testables Son aquellas que pueden expresarse mediante propiedades directamente evaluables por Echidna.

Algunos ejemplos incluyen:

- determinadas restricciones de acceso
- invariantes económicas
- comprobaciones relacionadas con balances
- validaciones de lógica de negocio.

Vulnerabilidades no testables Corresponden a aquellas situaciones cuya explotación requiere condiciones externas o escenarios complejos difíciles de reproducir automáticamente.

Entre ellas destacan:

- ataques de reentrancia
- determinados problemas de interacción entre contratos
- escenarios dependientes del contexto de despliegue.

En estos casos, EVMAudit conserva la información asociada a la vulnerabilidad y genera advertencias específicas, pero no intenta forzar una validación automática que pudiera producir resultados incorrectos.

Esta decisión se adoptó con el objetivo de evitar conclusiones erróneas y mantener un comportamiento conservador.

4.7.5. Integración con Echidna

Una vez generado el wrapper correspondiente, el módulo Runner ejecuta Echidna y recupera los resultados obtenidos durante el proceso de fuzzing.

La salida generada por la herramienta se procesa posteriormente para incorporarla al informe final.

De esta manera, EVMAudit consigue integrar tres técnicas de análisis diferentes:

- análisis estático
- ejecución simbólica
- fuzzing.

La combinación de estas aproximaciones permite aprovechar las fortalezas de cada una de ellas y compensar parcialmente sus limitaciones individuales.

!TODO: referencia cruzada a la Sección 2.X (análisis estático).

!TODO: referencia cruzada a la Sección 2.X (ejecución simbólica).

!TODO: referencia cruzada a la Sección 2.X (fuzzing).

4.7.6. Beneficios de la generación automática

La generación automática de propiedades aporta diversas ventajas:

- reducción del trabajo manual
- mayor reutilización de resultados

- integración transparente con Echidna
- facilidad para incorporar nuevas vulnerabilidades
- mejora del flujo general de análisis.

No obstante, la validación automática obtenida no sustituye la revisión manual realizada por el auditor, sino que constituye una capa adicional de apoyo.

4.9. 4.8.Gestion de errores

4.9.1. 4.8. Gestión de errores y robustez de la solución

Debido a la dependencia de herramientas externas y a la complejidad del proceso de análisis, durante el diseño de EVMAudit se prestó especial atención a la gestión de errores y a la robustez del sistema.

La existencia de múltiples componentes externos hace necesario disponer de mecanismos que permitan detectar y manejar situaciones anómalas sin comprometer el funcionamiento global de la herramienta.

4.8.1. Necesidad de una gestión centralizada

Durante el proceso de análisis pueden producirse diferentes tipos de errores:

- ausencia de herramientas instaladas
- incompatibilidades entre versiones
- contratos inválidos
- timeouts durante el análisis
- errores internos de las herramientas utilizadas
- problemas durante la generación de wrappers.

La gestión individual de cada una de estas situaciones complicaría considerablemente el mantenimiento del sistema.

Por este motivo se implementó una jerarquía de excepciones propia que centraliza el tratamiento de los errores.

4.8.2. Jerarquía de excepciones

La arquitectura implementada incorpora una clase base común a partir de la cual se derivan los distintos tipos de error utilizados por EVMAudit.

Esta aproximación permite distinguir claramente entre:

- errores de configuración
- errores de análisis
- errores producidos por herramientas externas.

La utilización de excepciones específicas simplifica la depuración y mejora la legibilidad del código.

!TODO: insertar diagrama UML de excepciones.

4.8.3. Detección de dependencias externas

Antes de ejecutar las herramientas integradas, EVMAudit verifica que las dependencias necesarias se encuentren correctamente instaladas.

Este mecanismo permite detectar problemas de configuración de forma temprana y proporcionar mensajes de error más descriptivos.

La validación previa evita que el proceso de análisis falle de forma inesperada en fases posteriores.

4.8.4. Gestión de timeouts

Algunas herramientas, especialmente aquellas basadas en ejecución simbólica, pueden presentar tiempos de análisis elevados dependiendo de la complejidad del contrato.

Con el objetivo de evitar bloqueos indefinidos, EVMAudit incorpora límites temporales para determinadas operaciones.

La utilización de timeouts permite:

- mejorar la estabilidad del sistema
- evitar consumos excesivos de recursos
- garantizar la finalización del análisis.

4.8.5. Preservación de resultados ante errores

Cuando se produce un fallo durante alguna fase del análisis, EVMAudit intenta conservar la información obtenida hasta ese momento.

Esta estrategia permite:

- facilitar la depuración
- conservar evidencias parciales
- evitar la pérdida completa del trabajo realizado.

La preservación de resultados resulta especialmente relevante en entornos de auditoría, donde incluso los análisis incompletos pueden aportar información útil.

4.8.6. Robustez frente a comportamientos específicos de las herramientas

Durante el desarrollo se detectaron determinados comportamientos particulares en las herramientas integradas.

Entre ellos destacan:

- códigos de retorno no convencionales
- formatos de salida inconsistentes
- diferencias entre versiones
- mensajes mezclados con datos estructurados.

Para hacer frente a estas situaciones se implementaron mecanismos específicos de adaptación y validación.

Estas medidas permitieron integrar las herramientas seleccionadas sin modificar su funcionamiento interno.

4.8.7. Beneficios de la estrategia adoptada

La incorporación de mecanismos de gestión de errores proporciona diversas ventajas:

- mayor estabilidad
- mejora de la experiencia de usuario

- facilidad de mantenimiento
- simplificación de la depuración
- incremento de la robustez general del sistema.

La existencia de una capa de gestión de errores independiente contribuye además a mantener desacoplados los distintos módulos que forman la arquitectura de EVMAudit.

!TODO: insertar referencia cruzada al módulo Runner.

!TODO: insertar referencia cruzada al diagrama de arquitectura general.

4.10. 4.9.Integración y distribución

4.10.1. 4.9. Distribución e integración de la solución

Uno de los objetivos perseguidos durante el desarrollo de EVMAudit fue diseñar una solución desacoplada de cualquier interfaz concreta y suficientemente flexible para poder ser reutilizada en distintos contextos.

Por este motivo, la lógica principal de análisis se implementó como una librería independiente, separando completamente las funcionalidades relacionadas con la detección y procesamiento de vulnerabilidades de los mecanismos de interacción con el usuario.

Esta decisión permitió demostrar la reutilización de la solución desarrollada mediante su integración en otros componentes software sin necesidad de modificar el núcleo de la herramienta.

4.9.1. Distribución de la librería

La implementación de EVMAudit como una librería independiente permite desacoplar el motor de análisis de cualquier entorno concreto de ejecución.

Este enfoque proporciona diversas ventajas:

- reutilización de la herramienta desde otros proyectos
- separación entre lógica de negocio e interfaces de usuario
- facilidad de mantenimiento
- posibilidad de incorporar nuevas interfaces en el futuro.

La distribución de la librería permite que el proceso de análisis pueda ejecutarse desde aplicaciones externas sin necesidad de duplicar funcionalidades.

!TODO: referencia cruzada al Anexo A (publicación en PyPI).

4.9.2. Aplicación web como caso de uso

Con el objetivo de validar la reutilización de la arquitectura desarrollada, se implementó una aplicación web que utiliza EVMAudit como motor de análisis.

La aplicación no implementa mecanismos propios de detección de vulnerabilidades, sino que delega completamente las tareas de análisis en la librería desarrollada.

Esta aproximación permite mantener una única implementación del proceso de análisis y garantiza la consistencia de los resultados obtenidos.

La aplicación actúa, por tanto, como una capa de presentación encargada de:

- recibir contratos inteligentes proporcionados por el usuario
- invocar las funciones de EVMAudit
- mostrar el progreso del análisis
- presentar los resultados obtenidos.

De esta forma, la aplicación web constituye una demostración práctica de la reutilización de la solución desarrollada.

4.9.3. Flujo de funcionamiento de la aplicación

La aplicación desarrollada permite dos mecanismos de entrada:

- carga de contratos Solidity mediante fichero
- introducción directa del código fuente a través de la interfaz web.

Una vez recibido el contrato, la aplicación inicia el proceso de análisis y ejecuta internamente el pipeline implementado por EVMAudit.

Las distintas fases del proceso se ejecutan de forma secuencial:

1. ejecución de Slither
2. ejecución de Mythril

3. normalización de resultados
4. correlación de vulnerabilidades
5. generación de propiedades
6. ejecución de Echidna
7. generación del informe final.

Durante la ejecución, la aplicación informa al usuario del progreso alcanzado y permite recuperar el informe una vez finalizado el análisis.

Este comportamiento demuestra que la arquitectura modular adoptada permite integrar EVMAudit en aplicaciones externas sin modificar la lógica interna del sistema.

!TODO: insertar diagrama simplificado de la aplicación web.

!TODO: insertar captura de pantalla de la interfaz.

!TODO: referencia cruzada al Anexo B (aplicación web).

4.9.4. Separación entre presentación y lógica de análisis

La decisión de implementar EVMAudit como una librería independiente permite mantener separadas las responsabilidades del sistema.

Mientras que la librería se encarga de:

- ejecutar herramientas externas
- procesar resultados
- correlacionar vulnerabilidades
- generar informes

la aplicación web únicamente proporciona los mecanismos de interacción con el usuario.

Esta separación favorece:

- la mantenibilidad del sistema
- la reutilización de la solución
- la incorporación de nuevas interfaces
- la evolución independiente de cada componente.

En consecuencia, la aplicación web debe entenderse como una demostración de la capacidad de integración de EVMAudit y no como la contribución principal del trabajo.

4.9.5. Consideraciones sobre infraestructura

Los aspectos relacionados con la publicación de la librería, el despliegue de la aplicación y la infraestructura utilizada no constituyen el núcleo de la contribución desarrollada en este Trabajo Fin de Máster.

Por este motivo, dichos elementos se incluyen en los anexos del documento.

!TODO: referencia cruzada al Anexo A (PyPI).

!TODO: referencia cruzada al Anexo B (Aplicación web).

!TODO: referencia cruzada al Anexo C (Docker).

!TODO: referencia cruzada al Anexo D (Despliegue).

4.11. 4.10. Flujo completo

4.11.1. 4.10. Flujo completo del análisis

Una vez descritos los distintos módulos que componen EVMAudit, resulta posible analizar el funcionamiento global de la solución.

El proceso completo implementado por la herramienta se compone de varias fases consecutivas que permiten transformar un contrato inteligente en un informe consolidado de vulnerabilidades.

4.10.1. Recepción del contrato

El proceso comienza con la recepción del contrato inteligente que se desea analizar.

El contrato puede proceder de distintos orígenes:

- ejecución directa de la librería
- aplicación web desarrollada
- futuras integraciones externas.

Tras recibir el contrato, EVMAudit detecta automáticamente el nombre del contrato y configura la versión adecuada del compilador Solidity.

4.10.2. Obtención de resultados mediante Slither y Mythril

La siguiente fase consiste en ejecutar las herramientas de análisis principales.

En primer lugar se ejecuta Slither, aprovechando las ventajas del análisis estático.

Posteriormente se ejecuta Mythril, incorporando las capacidades derivadas de la ejecución simbólica.

La combinación de ambas técnicas permite obtener una cobertura superior a la proporcionada por cada herramienta de forma individual.

!TODO: referencia cruzada a la Sección 2.X (herramientas de análisis).

4.10.3. Normalización de resultados

Las salidas generadas por las herramientas se transforman posteriormente a una estructura común.

Este proceso elimina las diferencias existentes entre ambas herramientas y permite trabajar con una representación homogénea de las vulnerabilidades.

La normalización constituye el paso previo necesario para aplicar el mecanismo de correlación.

4.10.4. Correlación de vulnerabilidades

Una vez normalizados los resultados, EVMAudit agrupa aquellas vulnerabilidades equivalentes detectadas por varias herramientas.

La correlación permite:

- reducir duplicidades
- aumentar la confianza de determinados hallazgos
- simplificar la interpretación del informe final.

Esta fase representa uno de los principales elementos diferenciadores de la solución desarrollada.

4.10.5. Generación de propiedades y fuzzing

A partir de las vulnerabilidades correlacionadas, el sistema consulta el catálogo SWC y genera automáticamente las propiedades necesarias para ejecutar Echidna.

Posteriormente se construye un wrapper Solidity específico y se realiza una fase adicional de validación mediante fuzzing.

Este proceso proporciona información complementaria sobre determinadas vulnerabilidades detectadas durante las fases anteriores.

4.10.6. Generación del informe final

Finalmente, toda la información obtenida durante el análisis es consolidada en un informe único.

El informe incluye:

- vulnerabilidades detectadas
- herramientas que las han identificado
- nivel de confianza asociado
- resultados del fuzzing
- metadatos adicionales.

De este modo, el auditor dispone de una visión unificada del estado de seguridad del contrato analizado.

4.10.7. Resumen del proceso

El flujo general implementado por EVMAudit puede resumirse en las siguientes etapas:

1. Recepción del contrato.
2. Configuración del entorno.
3. Ejecución de Slither.
4. Ejecución de Mythril.
5. Normalización.
6. Correlación.
7. Consulta del catálogo SWC.
8. Generación del wrapper.

9. Ejecución de Echidna.

10. Generación del informe.

La secuencia completa permite combinar distintas técnicas de análisis dentro de una arquitectura unificada y automatizada.

!TODO: insertar diagrama de secuencia del pipeline completo.

!TODO: insertar diagrama de flujo del proceso.

!TODO: insertar referencia a la Figura X.

!TODO: insertar referencia cruzada a las secciones 4.3 a 4.9.

4.12. 4.11. Evaluación experimental

4.12.1. 4.11. Flujo completo del análisis

Una vez descritos los distintos módulos que componen EVMAudit, resulta posible analizar el funcionamiento global de la solución.

El proceso completo implementado por la herramienta se compone de varias fases consecutivas que permiten transformar un contrato inteligente en un informe consolidado de vulnerabilidades.

4.11.1. Recepción del contrato

El proceso comienza con la recepción del contrato inteligente que se desea analizar.

El contrato puede proceder de distintos orígenes:

- ejecución directa de la librería
- aplicación web desarrollada
- futuras integraciones externas.

Tras recibir el contrato, EVMAudit detecta automáticamente el nombre del contrato y configura la versión adecuada del compilador Solidity.

4.11.2. Obtención de resultados mediante Slither y Mythril

La siguiente fase consiste en ejecutar las herramientas de análisis principales.

En primer lugar se ejecuta Slither, aprovechando las ventajas del análisis estático.

Posteriormente se ejecuta Mythril, incorporando las capacidades derivadas de la ejecución simbólica.

La combinación de ambas técnicas permite obtener una cobertura superior a la proporcionada por cada herramienta de forma individual.

!TODO: referencia cruzada a la Sección 2.X (herramientas de análisis).

4.11.3. Normalización de resultados

Las salidas generadas por las herramientas se transforman posteriormente a una estructura común.

Este proceso elimina las diferencias existentes entre ambas herramientas y permite trabajar con una representación homogénea de las vulnerabilidades.

La normalización constituye el paso previo necesario para aplicar el mecanismo de correlación.

4.11.4. Correlación de vulnerabilidades

Una vez normalizados los resultados, EVMAudit agrupa aquellas vulnerabilidades equivalentes detectadas por varias herramientas.

La correlación permite:

- reducir duplicidades
- aumentar la confianza de determinados hallazgos
- simplificar la interpretación del informe final.

Esta fase representa uno de los principales elementos diferenciadores de la solución desarrollada.

4.11.5. Generación de propiedades y fuzzing

A partir de las vulnerabilidades correlacionadas, el sistema consulta el catálogo SWC y genera automáticamente las propiedades necesarias para ejecutar Echidna.

Posteriormente se construye un wrapper Solidity específico y se realiza una fase adicional de validación mediante fuzzing.

Este proceso proporciona información complementaria sobre determinadas vulnerabilidades detectadas durante las fases anteriores.

4.11.6. Generación del informe final

Finalmente, toda la información obtenida durante el análisis es consolidada en un informe único.

El informe incluye:

- vulnerabilidades detectadas
- herramientas que las han identificado
- nivel de confianza asociado
- resultados del fuzzing
- metadatos adicionales.

De este modo, el auditor dispone de una visión unificada del estado de seguridad del contrato analizado.

4.11.7. Resumen del proceso

El flujo general implementado por EVMAudit puede resumirse en las siguientes etapas:

1. Recepción del contrato.
2. Configuración del entorno.
3. Ejecución de Slither.
4. Ejecución de Mythril.
5. Normalización.
6. Correlación.
7. Consulta del catálogo SWC.
8. Generación del wrapper.
9. Ejecución de Echidna.
10. Generación del informe.

La secuencia completa permite combinar distintas técnicas de análisis dentro de una arquitectura unificada y automatizada.

!TODO: insertar diagrama de secuencia del pipeline completo.

!TODO: insertar diagrama de flujo del proceso.

!TODO: insertar referencia a la Figura X.

!TODO: insertar referencia cruzada a las secciones 4.3 a 4.9.

4.11.X. Evaluación del contrato con una vulnerabilidad conocida: [NOMBRE DEL CONTRATO]

Descripción del contrato El contrato [NOMBRE DEL CONTRATO] fue seleccionado debido a que presenta una vulnerabilidad conocida relacionada con [TIPO DE VULNERABILIDAD].

Este contrato pertenece a [FUENTE DEL CONTRATO] y fue incluido en la evaluación con el objetivo de analizar el comportamiento de EVMAudit frente a escenarios representativos.

La vulnerabilidad esperada en este caso corresponde a:

- TODO.

!TODO: referencia cruzada a la Sección 2.X (descripción de la vulnerabilidad).

Resultados obtenidos por las herramientas En primer lugar, el contrato fue procesado individualmente por Slither y Mythril.

Los resultados obtenidos se resumen en la Tabla X.

[@@ Herramienta Vulnerabilidades detectadas

Slither TODO

Mythril TODO

!TODO: insertar referencia a la Tabla X.

Resultados tras la correlación Una vez aplicados los mecanismos de normalización y correlación implementados por EVMAudit, se obtuvo un total de **TODO** vulnerabilidades consolidadas.

La Tabla X muestra la relación entre los hallazgos originales y los resultados finales.

@@@ Métrica Valor

Hallazgos originales TODO

Hallazgos correlacionados TODO

Vulnerabilidades confirmadas TODO

Reducción de duplicidades TODO

!TODO: insertar referencia a la Tabla X.

Resultados del fuzzing mediante Echidna A partir de las vulnerabilidades obtenidas se generaron automáticamente las propiedades correspondientes para Echidna.

Los resultados obtenidos fueron los siguientes:

- Propiedades generadas: TODO.
- Propiedades ejecutadas correctamente: TODO.
- Vulnerabilidades verificadas: TODO.
- Limitaciones observadas: TODO.

!TODO: insertar captura o fragmento representativo del resultado de Echidna.

Discusión Los resultados obtenidos muestran que [**OBSERVACIÓN PRINCIPAL**].

En particular:

- TODO.
- TODO.
- TODO.

Este caso pone de manifiesto [**CONCLUSIÓN PRINCIPAL DEL CONTRATO**].

4.11.X. Evaluación del contrato con varias vulnerabilidades: [NOMBRE DEL CONTRATO]

Descripción del contrato El contrato [NOMBRE] incorpora una vulnerabilidad relacionada con [TIPO], lo que permite estudiar la capacidad de detección de las herramientas integradas cuando existen varias debilidades simultáneas.

!TODO: describir brevemente el funcionamiento del contrato.

Comparativa entre herramientas La Tabla X resume las vulnerabilidades detectadas por cada herramienta.

	@lcc@	Vulnerabilidad	Slither	Mythril
TODO	[X]	[X]		
TODO	[X]	[-]		
TODO	[-]	[X]		

Los resultados muestran que ambas herramientas presentan comportamientos complementarios.

Efecto del mecanismo de correlación La aplicación del proceso de correlación permitió:

- eliminar duplicidades;
- unificar nomenclaturas;
- aumentar la confianza de determinados hallazgos.

!TODO: insertar número real de vulnerabilidades correlacionadas.

Resultados del fuzzing El proceso de generación automática produjo un total de **TODO** propiedades.

Durante la ejecución de Echidna se observó:

- TODO.

!TODO: describir vulnerabilidades no testables.

Discusión Este contrato evidencia que las distintas técnicas de análisis empleadas por EVMAudit permiten obtener información complementaria y mejorar la cobertura respecto al uso individual de cada herramienta.

4.11.X. Evaluación del contrato real [NOMBRE DEL CONTRATO]

Descripción del contrato Además de los contratos vulnerables de referencia, se incluyó el contrato [NOMBRE], obtenido de [REPOSITORIO O FUENTE], con el objetivo de analizar el comportamiento de EVMAudit en un escenario más próximo a un entorno real.

!TODO: describir brevemente la finalidad del contrato.

Resultados obtenidos La Tabla X resume las vulnerabilidades detectadas durante el análisis.

[]@ll@	Métrica	Valor
--------	---------	-------

Vulnerabilidades detectadas por Slither	TODO
---	------

Vulnerabilidades detectadas por Mythril	TODO
---	------

Vulnerabilidades finales	TODO
--------------------------	------

Vulnerabilidades confirmadas	TODO
------------------------------	------

Observaciones sobre la correlación En este caso se observó que:

- TODO.

Asimismo, se identificaron diferencias entre ambas herramientas en relación con:

- TODO.
-

Resultados del fuzzing La fase de fuzzing permitió:

- TODO.

No obstante, algunas vulnerabilidades no pudieron validarse automáticamente debido a:

- TODO.

Discusión El análisis realizado demuestra que EVMAudit puede aplicarse también sobre contratos reales y no únicamente sobre ejemplos académicos diseñados específicamente para contener vulnerabilidades.

!TODO: indicar limitaciones observadas durante el análisis.

5. Conclusiones y trabajo futuro

5.1. 5. Conclusiones y trabajo futuro

5.1.1. 5.1 Conclusiones

5.1.2. 5.2 Limitaciones de la solución

- **Dependencia de herramientas externas.** EVMAudit depende del correcto funcionamiento de Slither, Mythril y Echidna. Cualquier modificación en dichas herramientas puede afectar al comportamiento del sistema.
- **Correlación basada en reglas.** La correlación implementada utiliza únicamente:
 - contrato
 - función
 - SWC

Esto puede provocar que determinadas vulnerabilidades relacionadas no se agrupen correctamente.

- **Falsos positivos y falsos negativos.** La herramienta no elimina completamente los falsos positivos inherentes a las herramientas integradas.
- **Limitaciones del fuzzing.** Determinadas vulnerabilidades, como la reentrancia, requieren contratos atacantes específicos y no pueden validarse automáticamente.
- **Ausencia de análisis inter-contrato.** Actualmente la herramienta analiza cada contrato de forma independiente.
- **Priorización limitada.** La priorización se basa en la severidad y en el número de herramientas que detectan la vulnerabilidad, sin utilizar métricas más avanzadas como CVSS.

5.1.3. 5.3 Líneas de trabajo futuro

- incorporar nuevas herramientas

- análisis inter-contrato
- CVSS
- Machine Learning
- Sengrep
- Manticore
- dashboard web

Referencias

A. Ejemplos de vulnerabilidades

A.1. ANEXO A. Ejemplos de vulnerabilidades en contratos inteligentes

Este anexo presenta ejemplos simplificados de vulnerabilidades representativas en contratos inteligentes desarrollados en Solidity. El objetivo es ilustrar de forma práctica los principales tipos de debilidades descritas en la sección 4.4, facilitando su comprensión y su posterior detección mediante herramientas automáticas.

Cada ejemplo incluye: descripción, fragmento de código vulnerable, impacto y estrategia de mitigación.

A.1.1. ANEXO A.1. Vulnerabilidades técnicas de ejecución

ANEXO A.1.1. Reentrancy

La vulnerabilidad de **reentrancy** se produce cuando un contrato realiza una llamada externa antes de actualizar su estado interno, permitiendo que el contrato receptor reingrese en la función original en un estado inconsistente.

Código vulnerable

```
[] contract VulnerableBank {
    mapping(address => uint256) public balances;
    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }
    function withdraw(uint256 amount) public {
        require(balances[msg.sender] >= amount);
        (bool success, ) = msg.sender.call{value: amount}();
        require(success);
        balances[msg.sender] -= amount;
    }
}
```

- **Impacto:** Un atacante puede drenar fondos repitiendo la llamada antes de que el balance sea actualizado.

- Mitigación:
 - Patrón *Checks-Effects-Interactions*
 - Uso de ReentrancyGuard
 - Actualizar el estado antes de llamadas externas

ANEXO A.1.2. Integer Overflow / Underflow

Errores aritméticos que provocan desbordamientos en operaciones enteras. Aunque mitigados en Solidity ≥ 0.8 , siguen siendo relevantes en bloques unchecked.

Código vulnerable

```
[] function increment(uint256 x) public pure returns (uint256) {  
    unchecked {  
        return x + 1;  
    }  
}
```

- **Impacto:** Puede alterar balances o condiciones lógicas críticas.
- Mitigación:
 - Evitar unchecked salvo casos justificados
 - Uso de validaciones explícitas

ANEXO A.1.3. Uso inseguro de delegatecall

La función delegatecall ejecuta código externo en el contexto de almacenamiento del contrato llamador.

Código vulnerable

```
[] contract Proxy {  
    address public implementation;  
    function execute(bytes memory data) public {  
        (bool success, ) = implementation.delegatecall(data);  
        require(success);  
    }  
}
```

- **Impacto:** Compromiso total del almacenamiento del contrato.
- Mitigación:
 - Control estricto de la dirección implementation
 - Uso de patrones proxy auditados (EIP-1967, UUPS)

ANEXO A.1.4. Denegación de servicio (DoS)

Bloqueo de ejecución debido a fallos en llamadas externas o estructuras no acotadas.

Código vulnerable

```
[] function payout(address[] memory recipients) public {  
    for (uint i = 0; i < recipients.length; i++) {  
        payable(recipients[i]).transfer(1 ether);  
    }  
}
```

- **Impacto:** Un solo fallo revierte toda la operación.
- Mitigación
 - Uso de patrón *pull over push*
 - Evitar bucles dependientes de input externo

A.1.2. ANEXO A.2. Vulnerabilidades técnicas de control y privilegios

ANEXO A.2.1. Falta de control de acceso

Funciones críticas accesibles por cualquier usuario.

Código vulnerable

```
[] contract Ownable {  
    address public owner;  
    function withdrawAll() public {  
        payable(msg.sender).transfer(address(this).balance);  
    }  
}
```

- **Impacto:** Pérdida total de fondos.

- Mitigación:
 - Uso de `onlyOwner`
 - Librerías como `OpenZeppelin AccessControl`

ANEXO A.2.2. Uso de `tx.origin`

Uso incorrecto de `tx.origin` para autenticación.

Código vulnerable

```
[] function withdraw() public {  
    require(tx.origin == owner);  
    payable(msg.sender).transfer(address(this).balance);  
}
```

- **Impacto:** Ataques mediante contratos intermediarios.
- **Mitigación:** Usar `msg.sender` para autenticación

ANEXO A.2.3. Inicialización insegura (contratos upgradeables)

En contratos upgradeables, la inicialización se realiza mediante funciones externas (`initialize`) en lugar de constructores. Si no están protegidas, cualquier usuario puede ejecutarlas y asumir el control del contrato.

Código vulnerable

```
[] function initialize(address _owner) public {  
    owner = _owner;  
}
```

- **Impacto:** Un atacante puede inicializar el contrato antes que el legítimo propietario.
- Mitigación:
 - Uso de `initializer` (`OpenZeppelin`)
 - Bloqueo de inicialización tras ejecución

A.1.3. ANEXO A.3. Vulnerabilidades económicas y del entorno

ANEXO A.3.1. Front-running / MEV

Un atacante observa la mempool y ejecuta transacciones antes que la víctima
A.

Código vulnerable

```
[] function buy(uint price) public {  
    require(price == currentPrice);  
    // compra  
}
```

- **Impacto:** Manipulación de operaciones (arbitraje, liquidaciones, subastas).
- Mitigación:
 - *Commit-reveal*
 - Subastas ciegas
 - Uso de *relayers* privados

ANEXO A.3.2. Dependencia de oráculos

Uso de datos externos manipulables.

Código vulnerable

```
[] function getPrice() public view returns (uint) {  
    return externalOracle.price();  
}
```

- **Impacto:** Manipulación de precios en DeFi.
- Mitigación:
 - Oráculos descentralizados (ej. Chainlink)
 - Promedios temporales (TWAP)

ANEXO A.3.3. Uso de `block.timestamp`

El uso de `block.timestamp` como fuente de aleatoriedad o para decisiones críticas es inseguro, ya que su valor puede ser parcialmente manipulado por mineros o validadores dentro de ciertos límites.

Código vulnerable

```
[] function random() public view returns (uint) {  
    return uint(keccak256(abi.encodePacked(block.timestamp)));  
}
```

- **Impacto:** Resultados predecibles o manipulables por mineros/validadores.
- Mitigación:
 - VRF (Verifiable Random Functions)
 - Fuentes externas verificables

A.1.4. ANEXO A.4. Errores lógicos de negocio

ANEXO A.4.1. Error en cálculo de balances

Errores en operadores lógicos o condiciones de validación pueden provocar inconsistencias en la gestión de balances, especialmente en casos límite donde las condiciones no cubren todos los escenarios posibles.

Código vulnerable

```
[] function withdraw(uint amount) public {  
    require(balances[msg.sender] > amount);  
    balances[msg.sender] -= amount;  
}
```

- **Impacto:** Comportamiento incorrecto en condiciones límite.
- Mitigación:
 - Uso de `>` en lugar de `>=`.

ANEXO A.4.2. Distribución incorrecta de recompensas

La lógica de distribución puede introducir errores debido a divisiones enteras o falta de gestión de restos, provocando pérdida de precisión y fondos no asignados correctamente.

Código vulnerable

```
[] function distribute() public {  
    uint reward = total / users.length;  
    for (uint i = 0; i < users.length; i++) {  
        balances[users[i]] += reward;  
    }  
}
```

■ Impacto:

- Pérdida de fondos debido a errores de redondeo (truncamiento en división entera)
- Acumulación de saldo no distribuido en el contrato
- Distribuciones injustas entre usuarios
- Posibles vectores de explotación si un atacante manipula el número de participantes

■ Mitigación:

- Uso de patrones de distribución que gestionen residuos (por ejemplo, acumuladores o *“remainder handling”*)
- Empleo de mayor precisión mediante escalado (*fixed-point arithmetic*)
- Validación de invariantes económicas (la suma distribuida debe coincidir con el total)
- Testing específico de casos límite (número de usuarios, valores pequeños, etc.)

ANEXO A.4.3. Estados inconsistentes

La falta de control adecuado sobre las transiciones de estado puede permitir la ejecución de funciones en condiciones no válidas, generando comportamientos inconsistentes en el contrato.

Código vulnerable

```

enum State { Open, Closed }

State public state;

function close() public {
    state = State.Closed;
}

function bid() public payable {
    require(state == State.Open);
}

```

- Impacto:

- Ejecución de funciones en estados no válidos
- Comportamiento inesperado del contrato
- Bloqueo o bypass de lógica de negocio
- Posible explotación combinada con otras vulnerabilidades (por ejemplo, *front-running* o *reentrancy*)

- Mitigación:

- Implementación de máquinas de estados explícitas y completas
- Uso de modificadores para validar estado (`inState(State.Open)`)
- Restricción de transiciones de estado válidas
- Aplicación de patrones *state machines*

A.1.5. ANEXO A.5. Resumen de vulnerabilidades

$$\begin{aligned} & \text{[]@ @p() * 0.0661 @p() * 0.1074 @p() * 0.1736 @p() * 0.1570 @p() * 0.0909 @p() *} \\ & 0.2479 @p() * 0.1570 @ \textbf{ID} \end{aligned}$$

Categoría

Vulnerabilidad

Tipo

Impacto

Detectable automáticamente

Ejemplo sección

- V1* Técnica Reentrancy Ejecución Crítico Sí (Slither/Mythril) ANEXO A.1.1
- V2* Técnica Overflow Aritmético Medio Sí (Slither) ANEXO A.1.2
- V3* Técnica Delegatecall Ejecución Crítico Parcial ANEXO A.1.3
- V4* Control Acceso no restringido Autorización Crítico Sí ANEXO A.2.1
- V5* Control tx.origin Autenticación Alto Sí ANEXO A.2.2
- V6* Económica Front-running MEV Alto No ANEXO A.3.1
- V7* Económica Oráculos Dependencia externa Crítico No ANEXO A.3.2
- V8* Lógica Error de balance Lógica Variable No ANEXO A.4.1

B. Entorno virtual Python

B.1. ANEXO B. Entorno virtual Python

El desarrollo de una librería Python destinada a integrar múltiples herramientas de análisis de seguridad plantea, desde el inicio, un problema de gestión de dependencias que no debe subestimarse. Las herramientas que se integran en la solución propuesta, como Slither, Mythril o Echidna, tienen requisitos de versión específicos y en ocasiones incompatibles entre sí cuando se instalan en el entorno global del sistema. Esta situación, conocida en el ecosistema Python como *dependency hell*, puede provocar conflictos silenciosos difíciles de depurar y comprometer la reproducibilidad del entorno de desarrollo.

Un entorno virtual (*virtual environment*) es un directorio aislado que contiene una instalación independiente del intérprete Python junto con sus propios paquetes y dependencias, sin interferir con el sistema global ni con otros proyectos. Esta separación proporciona varias ventajas fundamentales en el contexto del presente trabajo:

En primer lugar, garantiza el **aislamiento de dependencias**, de forma que cada proyecto mantiene sus propias versiones de bibliotecas sin afectar al resto del sistema. Esto resulta especialmente relevante cuando diferentes herramientas de análisis requieren versiones distintas de una misma dependencia transitiva.

En segundo lugar, favorece la **reproducibilidad del entorno de desarrollo**. Todos los integrantes del equipo pueden trabajar exactamente con las mismas versiones de todas las dependencias, eliminando la variabilidad asociada a instalaciones manuales y garantizando que los resultados obtenidos durante el desarrollo son consistentes independientemente del sistema operativo o configuración personal de cada desarrollador.

En tercer lugar, facilita el **ciclo de vida** del proyecto al delimitar claramente qué paquetes pertenecen al proyecto y cuáles son del sistema, simplificando tanto la distribución de la librería como su posterior publicación en registros públicos como PyPI.

B.1.1. ANEXO B.1. Tipos de entornos virtuales

En el ecosistema Python existen varias alternativas para la gestión de entornos virtuales, con distintos niveles de abstracción y funcionalidad.

El módulo estándar **venv**, incluido en la biblioteca estándar desde Python 3.3, permite crear entornos virtuales básicos mediante el comando `python -m venv .venv`. Sin embar-

go, este enfoque no incluye gestión de dependencias ni ficheros de bloqueo (*lockfiles*), por lo que debe complementarse con herramientas adicionales como `pip` y `pip-tools`.

`virtualenv` es una alternativa anterior al módulo estándar, con mayor compatibilidad con versiones antiguas de Python y algunas funcionalidades adicionales, aunque en la práctica ha quedado desplazada por las herramientas modernas.

`conda` ofrece un modelo más completo que combina gestión de entornos con gestión de paquetes, incluyendo dependencias no Python. Es habitual en entornos científicos y de análisis de datos, pero introduce una complejidad y un tamaño innecesarios para un proyecto centrado en el ecosistema Python puro.

Herramientas como `poetry` o `pipenv` representan un nivel superior de abstracción, combinando la gestión de entornos virtuales con la resolución de dependencias, la generación de ficheros de bloqueo y el ciclo de publicación de paquetes. Su adopción en proyectos profesionales se ha generalizado en los últimos años.

Finalmente, `uv` constituye la herramienta de última generación en este espacio, combinando todas las funcionalidades anteriores en una solución de rendimiento muy superior, como se detalla en la sección siguiente.

B.1.2. ANEXO B.2. Herramienta a usar: `uv`

`uv` es un gestor de paquetes y proyectos Python de alto rendimiento desarrollado por Astral, la empresa creadora del formateador Ruff. Implementado en Rust, se presenta como una herramienta unificada capaz de sustituir a `pip`, `pip-tools`, `pipx`, `poetry`, `pyenv`, `twine` y `virtualenv` mediante una interfaz de línea de comandos coherente. Su desarrollo se orienta tanto a la velocidad de ejecución como a la corrección en la resolución de dependencias y a la facilidad de adopción en proyectos de diferente escala y complejidad.

ANEXO B.2.1. Ventajas de `uv`

La principal ventaja diferencial de `uv` respecto a las alternativas existentes es su rendimiento. Según los propios *benchmarks* publicados en su documentación oficial, `uv` es entre 10 y 100 veces más rápido que `pip` en operaciones de instalación de paquetes con caché caliente. Esta diferencia resulta perceptible en la práctica, especialmente durante las fases de incorporación de nuevos integrantes al equipo o en entornos de integración continua donde el entorno debe reconstruirse frecuentemente.

Más allá del rendimiento, `uv` ofrece un conjunto de ventajas relevantes para el desarrollo de la librería propuesta en este trabajo. Gestiona automáticamente los entornos virtuales asociados a cada proyecto, sin necesidad de crearlos ni activarlos manualmente. Genera y mantiene un fichero de bloqueo universal (`uv.lock`) que garantiza instalaciones reproducibles en cualquier plataforma. Permite gestionar múltiples versiones del intérprete Python e instalar la versión adecuada de forma automática si no está disponible en el sistema. Además, su diseño es compatible con los estándares del ecosistema Python (`pyproject.toml`, PEP 517, PEP 621), lo que facilita la integración con otras herramientas y la publicación en registros de paquetes.

ANEXO B.2.2. Qué proporciona `uv` en el contexto de este proyecto

Para el desarrollo de la librería propuesta, `uv` proporciona un conjunto de funcionalidades que cubren todo el ciclo de vida del proyecto, desde la inicialización hasta la publicación.

Gestión de dependencias y sincronización del entorno. Una vez definidas las dependencias del proyecto en el fichero `pyproject.toml`, cualquier integrante del equipo puede reproducir exactamente el mismo entorno ejecutando un único comando:

```
[] uv sync
```

Este comando resuelve las dependencias declaradas, instala las versiones exactas registradas en el fichero de bloqueo `uv.lock` y configura el entorno virtual del proyecto de forma automática. La simplicidad de este flujo elimina los problemas habituales de divergencia entre entornos de desarrollo individuales, garantizando que todos los miembros del equipo trabajan con las mismas versiones de Slither, Mythril, Echidna y el resto de dependencias de la librería.

Inicialización de proyectos. `uv` proporciona soporte integrado para la creación de nuevos proyectos mediante el comando `uv init`. Para el caso específico de una librería Python destinada a ser importada por otros proyectos o publicada en PyPI, se utiliza la opción `--lib`:

```
[] uv init --lib evmaudit
```

Este comando genera automáticamente la estructura de directorios recomendada para una librería Python, incluyendo el fichero `pyproject.toml` con los metadatos del proyecto, el directorio `src/` con el paquete principal y los ficheros de configuración necesarios para la construcción y distribución. El uso de la disposición `src/` (*src layout*) es la práctica

recomendada actualmente para proyectos publicables, ya que evita problemas habituales relacionados con la importación del paquete desde el directorio raíz durante el desarrollo.

Construcción de distribuciones. `uv` integra soporte nativo para la generación de distribuciones instalables mediante el comando `uv build`, que produce tanto el archivo fuente (*sdist*) como la rueda binaria (*wheel*) del paquete:

```
[] uv build
```

El resultado son los artefactos estándar de distribución Python ubicados en el directorio `dist/`, listos para ser publicados o distribuidos directamente.

Publicación de paquetes. El ciclo se completa con el soporte para publicación en registros de paquetes, incluyendo PyPI, mediante el comando `uv publish`:

```
[] uv publish
```

Este comando gestiona la autenticación y la subida de los artefactos generados, cubriendo el flujo completo que anteriormente requería herramientas adicionales como `twine`.

En conjunto, `uv` unifica en una sola herramienta todo el ciclo de vida del proyecto: inicialización, gestión de dependencias, sincronización del entorno, construcción de distribuciones y publicación. Esta integración reduce la fricción en el desarrollo colaborativo y facilita la adopción de prácticas profesionales de gestión de proyectos Python desde las primeras fases del trabajo.

C. Distribución mediante PyPI

C.1. ANEXO C. DISTRIBUCIÓN Y PUBLICACIÓN EN EL REGISTRO DE PAQUETES PYPI

El ciclo de desarrollo de la librería propuesta culmina con su fase de distribución, permitiendo que las herramientas de análisis de seguridad implementadas sean accesibles e integrables por la comunidad de desarrollo y auditoría de *smart contracts*. Para asegurar una distribución estandarizada y eficiente dentro del ecosistema Python, se ha seleccionado el índice oficial de paquetes PyPI (*Python Package Index*). La gestión de este proceso se unifica bajo la herramienta `uv`, garantizando la consistencia desde la compilación de los artefactos hasta su publicación definitiva. ## ANEXO C.1. Configuración de Metadatos del Proyecto (`pyproject.toml`)

El paso previo indispensable para la distribución consiste en la definición inequívoca de los metadatos y la especificación del sistema de construcción (*build system*) en el archivo de configuración `pyproject.toml`, ubicado en la raíz del paquete. Este procedimiento se rige bajo los estándares modernos de empaquetado de Python (PEP 517 y PEP 621).

Para este proyecto, se ha adoptado `hatchling` como *build backend*, debido a su ligereza, velocidad y compatibilidad nativa con las especificaciones del ecosistema actual. A continuación, se presenta la estructura de configuración requerida para delimitar las propiedades de la librería `evmaudit`:

```
[project] name = .evmaudit
version = "0.1.0"
description = .Add your description here
readme = README.md
authors = [ { name = "Daniel Rovira Martínez"
, email = "pardalaco@gmail.com"
}, { name = "Paula Suárez Prieto"
, email = "Paula Suárez Prieto"
}, { name = .Adrián Moreno Martín
, email = .adrimore2@gmail.com
} ] requires-python = ">=3.12"
```



```
dependencies = [ .eth>=0.0.1
, "mythril>=0.24.8
, "setuptools<70.0.0
, "slither-analyzer>=0.9.2
, "solc-select>=1.2.0
,]
[project.scripts] evmaudit = .evmaudit.main:main
[build-system] requires = [üv_build>=0.11.15,<0.12.0
] build-backend = üv_build
```

Los clasificadores (*classifiers*) incluidos permiten categorizar la librería dentro del índice público, facilitando su indexación en función de la licencia de código abierto seleccionada, la compatibilidad del sistema operativo y las versiones soportadas del intérprete Python.

C.1.1. ANEXO C.2. Proceso de Compilación del Paquete

Una vez validados los metadatos, se procede a la generación de los archivos de distribución. Este proceso transforma el código fuente estructurado en el directorio `src/` en artefactos instalables e independientes del entorno de desarrollo.

La ejecución del comando unificado de compilación abstrae la complejidad de las herramientas tradicionales:

```
[] uv build
```

Este comando genera de forma nativa dos tipos de distribuciones estándar dentro del directorio `dist/`:

- **Distribución de código fuente (*sdist* o *.tar.gz*):** Un archivo comprimido que contiene el código fuente original y los archivos de configuración, actuando como respaldo para plataformas o configuraciones no previstas.
- **Distribución binaria compilada (*wheel* o *.whl*):** El formato de empaquetado moderno que permite una instalación directa y optimizada en el sistema destino, evitando la necesidad de compilar el código en la máquina del usuario final.

En contextos de desarrollo complejos donde el paquete forma parte de un repositorio principal o arquitectura de *workspace* (como el entorno de trabajo TFM-UNIR), `uv` permite

gestionar la compilación de manera localizada. Para forzar la construcción exclusiva del subproyecto desde su propio directorio y evitar colisiones en la raíz global, se aplica la opción de empaquetado específico:

```
[] uv build --package evmaudit
```

C.1.2. ANEXO C.3. Seguridad y Autenticación en la Publicación

La publicación de código en repositorios públicos exige mecanismos estrictos de control de acceso para prevenir vectores de ataque basados en la cadena de suministro (*supply chain attacks*). Por razones de seguridad, se desestima el uso de contraseñas de usuario tradicionales en favor de la autenticación basada en **Tokens de API**.

El proceso de despliegue requiere la obtención de un *token* con prefijo **pypi-** generado desde el panel de control de PyPI. En la primera interacción, el alcance del *token* se configura de manera global; no obstante, una vez realizada la primera subida con éxito, la buena práctica metodológica dicta restringir los permisos del token de manera exclusiva al ámbito del paquete **evmaudit**, minimizando así la superficie de exposición en caso de compromiso de la credencial.

C.1.3. ANEXO C.4. Ejecución del Despliegue

Con los artefactos ubicados en el directorio **dist/** y las credenciales expedidas, se procede a la transferencia segura hacia los servidores de PyPI. El comando **uv publish** automatiza la verificación de integridad mediante *hashes* criptográficos y realiza la subida en un único paso:

```
[] uv publish
```

Durante el flujo interactivo en la línea de comandos, el sistema requiere la introducción del identificador genérico **__token__** en el campo de usuario, seguido de la clave alfanumérica del token de API en el campo de contraseña. Con el objetivo de optimizar este flujo en entornos de Integración Continua (CI/CD) o evitar la inserción manual recurrente, es posible exportar temporalmente la credencial en el entorno de la terminal actual:

```
[] export UV_PUBLISH_TOKEN="pypi-tu-token-aqui  
uv publish
```

Tras la finalización exitosa del proceso, el paquete queda registrado globalmente, permitiendo su incorporación inmediata en otros proyectos mediante los gestores tradicionales del ecosistema:

```
[] pip install evmaudit
```

O bien, aprovechando los beneficios de rendimiento de la herramienta unificada del proyecto:

```
[] uv add evmaudit
```

C.1.4. ANEXO C.5. Ciclo de Mantenimiento y Actualización de Versiones

La evolución de la librería para la corrección de vulnerabilidades o la integración de nuevas capacidades de análisis requiere una política estricta de control de versiones. El flujo metodológico establecido para la liberación de actualizaciones iterativas consta de tres fases secuenciales: - **Incremento del número de versión:** Modificación manual del campo `version` en el archivo `pyproject.toml` siguiendo el estándar de Versionado Semántico (ej. de 0.1.0 a 0.1.1). - **Saneamiento del directorio de distribución:** Eliminación de los artefactos obsoletos del directorio `dist/` para mitigar el riesgo de duplicidad o subidas erróneas de versiones previas. - **Reconstrucción y despliegue:** Ejecución consecutiva de los procesos de empaquetado y transferencia:

```
[] uv build uv publish
```

Esta sistemática asegura que cada iteración de la herramienta de auditoría de la EVM mantenga la trazabilidad, la coherencia histórica y la disponibilidad pública necesarias para un entorno de producción académica y profesional.

D. Docker

D.1. Anexo D Docker

D.1.1. ANEXO D.1. CONTENEDORIZACIÓN E INFRAESTRUCTURA DE DESPLIEGUE (DOCKER)

En el ámbito del desarrollo de software moderno y la ciberseguridad, la reproducibilidad del entorno de ejecución constituye un pilar crítico. Tradicionalmente, el despliegue de aplicaciones que integran múltiples herramientas de análisis (como compiladores de Solidity y motores de ejecución simbólica) se enfrentaba al problema de “funciona en mi máquina”, derivado de las discrepancias en las versiones de las dependencias, librerías del sistema operativo y configuraciones locales. Para mitigar este riesgo, el presente proyecto adopta una arquitectura basada en contenedores de aplicación a través del ecosistema de **Docker** y **Docker Compose**.

D.1.2. ANEXO D.2. Fundamentos de Contenedorización: Docker y Docker Compose

Docker es una plataforma de código abierto basada en la tecnología de contenedorización, la cual permite empaquetar una aplicación y todas sus dependencias (binarios, librerías, archivos de configuración) en una unidad estandarizada denominada **contenedor**. A diferencia de la virtualización tradicional, Docker opera mediante la virtualización a nivel de sistema operativo, compartiendo el núcleo (kernel) del sistema anfitrión pero ejecutando los procesos en espacios de usuario completamente aislados a través de namespaces y cgroups. Desde la perspectiva de la seguridad, este aislamiento garantiza que los procesos del pipeline de auditoría de EVMAudit se ejecuten de forma confinada, mitigando el impacto en la infraestructura anfitriona ante la eventual ejecución de código arbitrario o inesperado durante el análisis de contratos inteligentes.

Por su parte, **Docker Compose** es la herramienta diseñada para definir y orquestar aplicaciones Docker multi-contenedor. Mediante el uso de un archivo de configuración declarativo en formato YAML (docker-compose.yml), permite definir con precisión los servicios que componen el sistema, sus dependencias de arranque, la exposición de puertos hacia el exterior, la creación de redes aisladas y la asignación de volúmenes persistentes.

En el contexto de EVMAudit, actúa como el motor de despliegue unificado, permitiendo al administrador inicializar toda la infraestructura del TFM (servidor FastAPI, interfaz web y almacenamiento de informes) de manera centralizada.

D.1.3. ANEXO D.3. Estrategia de Construcción de la Imagen (Dockerfile)

La construcción de la imagen se define en un único Dockerfile optimizado. Debido a que el pipeline de análisis requiere interactuar con el sistema operativo para invocar compiladores y binarios de seguridad, se ha seleccionado **Ubuntu 22.04** como imagen base, proporcionando un entorno estable y con soporte extendido para dependencias nativas de Linux en arquitectura amd64.

El proceso de aprovisionamiento de la imagen se divide en las siguientes fases críticas: 1.

Entorno y Variables de Sistema: Se configuran las variables de entorno `PYTHONDONTWRITEBYTECODE=1` y `PYTHONUNBUFFERED=1` para optimizar la ejecución de Python dentro del contenedor, evitando la escritura de residuos binarios y forzando el volcado de logs en tiempo real. Asimismo, se establece `DEBIAN_FRONTEND=noninteractive` para suprimir diálogos interactivos durante la instalación de paquetes. 2. **Aprovisionamiento de Compiladores (Solidity):** Se añade el repositorio PPA oficial de Ethereum (`ppa:ethereum/ethereum`) para incorporar el compilador nativo de Solidity (`solc`). Posteriormente, se instala la utilidad `solc-select` mediante el gestor de paquetes de Python para automatizar la descarga y conmutación de versiones. 3. **Integración del Fuzzer Echidna:** Dado que Echidna se distribuye de manera óptima como un binario estático para Linux, el contenedor automatiza su descarga directa (versión v2.3.2) desde los repositorios oficiales de *Cryptic*, procediendo a su extracción e instalación en `/usr/local/bin/` para garantizar su disponibilidad inmediata en el PATH del sistema. 4. **Optimización de Dependencias con uv (Multi-stage Build):** Con el objetivo de minimizar los tiempos de construcción y asegurar una gestión eficiente de los paquetes de Python, se emplea un mecanismo de construcción en etapas múltiples (Multi-stage build), importando los binarios optimizados del gestor uv directamente desde su imagen oficial en el registro de GitHub (`ghcr.io/astral-sh/uv:latest`). 5. **Instalación del Paquete Local:** Tras establecer el directorio de trabajo en `/app` y copiar el código fuente, se ejecuta el comando `uv sync --frozen --no-cache`. Esto resuelve de forma determinista el grafo de dependencias del archivo `uv.lock`, registrando el paquete local editable `evmaudit` e instalando el servidor ASGI Uvicorn sin almacenar

datos residuales en la caché de la imagen.

D.1.4. ANEXO D.4. Orquestación de Servicios (Docker Compose)

La coordinación del contenedor web y sus dependencias con el sistema anfitrión se gestiona de forma declarativa mediante un archivo `docker-compose.yml`. La especificación del servicio, denominado `evmaudit-web`, se fundamenta en tres pilares de ingeniería:

- **Persistencia de Datos mediante Volúmenes:** Con el fin de dotar al sistema de un estado persistente (pese a la naturaleza efímera de los contenedores), se realiza un mapeo directo de directorios del host hacia el contenedor:
- `./jsons/_uploads:/app/jsons/_uploads:` Almacena de forma persistente los contratos Solidity cargados por los usuarios, los wrappers intermedios generados para Echidna y los informes de auditoría finales en formato JSON y Markdown.
- `./contracts:/app/contracts:` Habilita un volumen opcional para la auditoría directa de Smart Contracts locales sin necesidad de interactuar con la interfaz web.
- **Aislamiento de Red y Mapeo de Puertos:** Se expone el puerto 8080 del contenedor hacia el puerto 8080 del sistema anfitrión. Esto permite redirigir el tráfico HTTP de la interfaz construida en HTML5/Vanilla JS hacia el backend desarrollado en FastAPI de forma transparente.
- **Tolerancia a Fallos:** Se implementa la política de reinicio `restart: unless-stopped`. Esta directiva asegura la alta disponibilidad del servicio ante excepciones imprevistas en el motor de ejecución simbólica (Mythril) o caídas del propio demonio de Docker, garantizando que el servicio web vuelva a levantarse de manera automática salvo interrupción explícita del administrador.

D.1.5. ANEXO D.5. Flujo de Despliegue y Ciclo de Vida del Contenedor

Para la puesta en marcha de la infraestructura local en entornos de desarrollo o evaluación, el ciclo de vida del contenedor se administra mediante el estándar de comandos de Docker Compose: 1. **Fase de Construcción (Build):** Compilación de la imagen e instalación del entorno virtual determinista:

```
[] docker compose build
```

2. **Fase de Inicialización (Up):** Despliegue e instanciación del servicio en segundo plano (detached mode):

```
[] docker compose up -d
```

3. **Fase de Auditoría de Ejecución (Logs):** Inspección de la salida estándar del contenedor para la monitorización de los análisis en curso:

```
[] docker compose logs -f evmaudit-web
```

4. **Fase de Parada (Down):** Interrupción y eliminación de los contenedores activos salvaguardando la integridad de los datos de las auditorías gracias a los volúmenes enlazados: `docker compose down`

E. Aplicación web

F. CI/CD

F.1. ANEXO F. ENTORNO DE INTEGRACIÓN CONTINUA (CI/CD) Y PUBLICACIÓN AUTOMATIZADA

Para garantizar la integridad del software durante el ciclo de vida del desarrollo y agilizar el flujo de despliegue, se ha diseñado e implementado un pipeline de Integración Continua (CI) basado en **GitHub Actions**. Esta estrategia de ingeniería de software permite automatizar la compilación, verificación y empaquetado de la aplicación en cada iteración, mitigando los riesgos asociados a la integración manual de código y asegurando la disponibilidad inmediata de artefactos listos para producción.

F.1.1. ANEXO F.1. Arquitectura del Workflow y Disparadores (Triggers)

El flujo de trabajo automatizado se define de manera declarativa mediante la sintaxis YAML de GitHub Actions. Con el propósito de optimizar los recursos de cómputo y mantener un control estricto sobre la estabilidad de la rama principal, se han configurado dos disparadores específicos:

- **Eventos de Empuje (*Push*):** El pipeline se ejecuta de forma automática ante cualquier consolidación directa de código en la rama `main`, asegurando que cada incremento de software sea evaluado y empaquetado de inmediato.
- **Solicitudes de Extracción (*Pull Requests*):** La automatización actúa como una barrera de calidad (*quality gate*) ante cualquier intento de fusión hacia la rama `main`. Esto permite validar que las modificaciones propuestas por los desarrolladores no rompan el proceso de construcción del contenedor antes de que el código sea integrado definitivamente.

El entorno de ejecución seleccionado para los trabajos (*jobs*) es `ubuntu-latest`, lo que proporciona un entorno virtual limpio, aislado y actualizado de manera nativa por la infraestructura de GitHub.

F.1.2. ANEXO F.2. Desglose Técnico de las Etapas del Pipeline

El ciclo de vida del pipeline se divide en dos trabajos secuenciales y dependientes, los cuales ejecutan tareas críticas de aprovisionamiento, autenticación y despliegue:

ANEXO F.2.1. Trabajo de Construcción y Publicación (`create-docker-image`)

Es el núcleo técnico de la automatización y consta de los siguientes pasos detallados:

1. **Clonación del Repositorio y Gestión de Submódulos:** Se utiliza la acción oficial `actions/checkout@v2`. Debido a la arquitectura desacoplada del proyecto, es indispensable configurar el parámetro `submodules: 'recursive'`. Esta directiva instruye al agente para que descargue e integre de manera automática el repositorio y código de `evmaudit` dentro de la estructura de directorios del pipeline.
2. **Autenticación en el Registro de Contenedores (GHCR):** Mediante la acción `docker/login-action@v2`, el pipeline establece una conexión segura con el registro oficial de GitHub (`ghcr.io`). El proceso se autentica de forma dinámica utilizando el actor del ciclo de vida (`${{ github.actor }}`) y un token de acceso seguro almacenado de forma cifrada en los secretos del repositorio bajo la clave `${{ secrets.IMAGES_TFM_UNIR }}`.
3. **Normalización del Espacio de Nombres y Doble Etiquetado (*Multi-tagging*):**
Las especificaciones de Docker y el estándar de la Open Container Initiative (OCI) prohíben estrictamente el uso de caracteres en mayúscula para los nombres de las imágenes de contenedores. Para solucionar esto, el pipeline ejecuta un script en Bash que convierte dinámicamente el nombre del repositorio a minúsculas utilizando el comando `tr '[:upper:]' '[:lower:]'`. Posteriormente, se implementa una estrategia de doble etiquetado para optimizar la trazabilidad y la inmutabilidad:
 - **Etiqueta por SHA (IMAGE_SHA):** Vincula de forma unívoca el contenedor con el hash del *commit* específico de Git que originó la compilación (`${{ github.sha }}`). Esto permite realizar auditorías retrospectivas y despliegues deterministas en caso de fallos.
 - **Etiqueta de Última Versión (IMAGE_LATEST):** Sobrescribe el puntero `:latest` con la versión más reciente del software que haya superado la fase de construcción con éxito.

4. **Construcción y *Push* Multiplataforma:** Se invoca de manera directa el comando `docker build`, forzando la compilación bajo la arquitectura destino `--platform linux/amd64` utilizando el `Dockerfile` del proyecto como plano de construcción. Una vez generadas las imágenes locales con sus respectivas etiquetas, se ejecutan las instrucciones de empuje (*push*) hacia el registro seguro de GitHub, quedando el artefacto disponible para su consumo.

ANEXO F.2.2. Trabajo de Despliegue (deploy) y Limitaciones del Entorno

Para garantizar una separación formal de conceptos en la arquitectura de la integración, el pipeline implementa un segundo trabajo secuencial denominado `deploy`. Utilizando la directiva `needs: create-docker-image`, se establece una restricción de dependencia estricta: esta etapa no puede inicializarse si el empaquetado y la publicación previa de la imagen en el registro han fallado.

En la fase actual del proyecto, **esta etapa automatizada no ha sido implementada de forma activa y opera estrictamente como un punto de anclaje (*placeholder*) arquitectónico**. La justificación de esta decisión de diseño radica en las limitaciones técnicas del entorno de alojamiento seleccionado para las pruebas de concepto. La infraestructura de EVMAudit se despliega externamente en la plataforma PaaS **Railway** utilizando su modalidad de suscripción gratuita. Este nivel de servicio impone restricciones en las interfaces de programación (APIs) y en el uso de *webhooks*, impidiendo la ejecución de despliegues totalmente automatizados (*Automated CD Triggers*) desencadenados de forma directa mediante agentes de terceros como GitHub Actions.

Por consiguiente, el flujo de trabajo en este punto se limita a verificar la integridad de la secuencia de comandos en consola, quedando el aprovisionamiento de la infraestructura supeditado a los mecanismos nativos de la plataforma de destino. En el **siguiente apartado (X.5. Despliegue de la Infraestructura)**, se expondrá de manera más amplia la configuración, el aprovisionamiento y las características operativas de dicho entorno en Railway.

G. Railway

G.1. ANEXO G. DESPLIEGUE DE LA INFRAESTRUCTURA EN LA NUBE (RAILWAY)

Para validar la operatividad de EVMAudit en un entorno accesible y simular un escenario de producción real, se ha procedido al despliegue de la arquitectura contenedorizada en la plataforma de Plataforma como Servicio (PaaS) **Railway**. A continuación, se detallan las especificaciones del entorno, las restricciones técnicas de hardware identificadas y las optimizaciones de ingeniería aplicadas en el código fuente para garantizar la estabilidad del sistema.

G.1.1. ANEXO G.1. Aprovisionamiento y Configuración del Entorno Cloud

El proceso de despliegue en la infraestructura de la nube se ha estructurado bajo las siguientes directrices operativas:

- **Selección del Nivel de Servicio:** La instancia se ha instanciado haciendo uso del nivel gratuito (*Free Tier*) de la plataforma, el cual provee un crédito base de \$5 USD o un límite temporal de 30 días de cómputo.
- **Aislamiento Regional:** Con el objetivo de minimizar la latencia de red en las peticiones HTTP y optimizar la transferencia de datos, se ha seleccionado la región europea con nodo central en **Ámsterdam (EU)**.
- **Mapeo de Infraestructura y Orquestación:** El aprovisionamiento se realiza directamente vinculando el contenedor web a la imagen Docker compilada y almacenada en el registro de GitHub (ghcr.io), exponiendo de manera transparente la API del *backend* desarrollada en FastAPI.
- **Enrutamiento y Capa de Enlace (SSL):** La plataforma genera de manera dinámica un nombre de dominio completamente cualificado (FQDN) provisto de seguridad criptográfica TLS/SSL (HTTPS) para el acceso público a la interfaz de usuario: `https://evmaudit-production.up.railway.app/`.

G.1.2. ANEXO G.2. Limitaciones de Hardware y el Problema del Desbordamiento de Memoria (OOM)

La modalidad gratuita de la plataforma PaaS impone restricciones estrictas sobre los recursos de hardware asignados a cada contenedor, parametrizados de la siguiente forma:

- **Capacidad de Cómputo (CPU):** 2 vCPU virtuales compartidas.
- **Memoria Volátil (RAM):** 1 GB con un límite estricto de cuota (*Plan Limit*).

Bajo un escenario de despliegue convencional en servidores dedicados o infraestructura local, el pipeline de análisis de EVMAudit se ejecuta sin restricciones debido a la disponibilidad de memoria elástica. Sin embargo, en el entorno de la nube restringido, el motor de *fuzzing* basado en propiedades **Echidna** presenta un problema crítico de arquitectura.

Echidna, al estar desarrollado en Haskell, requiere de forma nativa una reserva inicial y un espacio de intercambio que supera con creces el gigabyte de memoria RAM para gestionar el mapa de cobertura del binario y la generación de casos de prueba. Al alcanzar el umbral crítico de 1 GB asignado por Railway, el demonio del sistema operativo anfitrión (*Kernel Out-of-Memory Killer*) destruía de manera abrupta el contenedor para salvaguardar la integridad del nodo, provocando la caída del servicio web y reportando un error de tipo *OOM*.

G.1.3. ANEXO G.3. Optimización del Sistema en Tiempo de Ejecución (RTS) de Haskell

Para mitigar el desbordamiento de memoria sin alterar las capacidades analíticas esenciales de la herramienta, se realizó una intervención a nivel de código en el módulo de control del *pipeline* (`run_echidna`). La solución consistió en inyectar directivas específicas orientadas a reconfigurar los parámetros del **Runtime System (RTS)** del compilador de Glasgow Haskell (GHC) empaquetados dentro del binario de Echidna.

La estructura de invocación del proceso fue modificada e implementada en Python mediante el siguiente diseño de argumentos:

```
[] command = [ .echidna  
, contract_path, -contract  
, contract_name, -format  
, "json
```

```
,  
# Reducción del espacio de búsqueda del Fuzzer -test-limit  
, "100  
,  
# Optimización de la compilación inicial -solc-args  
, -optimize-runs 0  
,  
# Apertura de las opciones del Runtime System de Haskell -RTS  
,  
# Parámetros estrictos de control de memoria y concurrencia M950m  
, c  
, N1  
,  
# Cierre de las opciones RTS RTS  
]
```

A continuación se expone la justificación técnica detrás de los modificadores inyectados:

- **Restricción de Ensayos (--test-limit 100):** Al parametrizar el límite de pruebas en 100 iteraciones, se acota la profundidad del grafo de ejecución generado por el fuzzer. Esto reduce el consumo de memoria acumulativo a lo largo del tiempo de computación.
- **Techo Infranqueable de Memoria (-M950m):** Esta directiva establece que el asignador de memoria de Haskell tiene prohibido estrictamente reclamar más de 950 megabytes del espacio de usuario. Al situar este límite ligeramente por debajo del gigabyte real de Railway, se evita que el sistema operativo de la nube elimine el proceso por exceder la cuota física.
- **Recolección de Basura Agresiva (-c):** Activa un algoritmo de recolección de residuos más severo en el recolector de basura de Haskell (*Garbage Collector*). En lugar de acumular objetos intermedios en la memoria RAM, el sistema libera los recursos obsoletos inmediatamente después de cada evaluación de propiedad.
- **Concurrencia Confinada (-N1):** Limita la ejecución del entorno de ejecución a un único hilo de procesamiento, evitando la duplicación de estructuras de datos en memoria asociadas al paralelismo de hilos nativos.

Es importante destacar que la optimización descrita no se integró de forma estática en la construcción del archivo Dockerfile (lo que habrí alterado el comportamiento de la imagen base de manera permanente), sin que se aplicó directamente sobre el código fuente desplegado en el contenedor en ejecución (*runtime*). Bajo condiciones de despliegue convencionales en infraestructuras con escalabilidad elástica o recurso dedicados de hardware, esta intervención técnica resultaría completamente innecesaria, ya que la aplicación contaría con la memoria suficiente para procesar el *pipeline* por defecto. Por consiguiente, esta modificación responde de manera estricta a un mecanismo de mitigación ad hoc, implementado exclusivamente para sortear las limitaciones físicas del entorno gratuito y evitar la interrupción forzada del servicio web por falta de memoria.

Conclusión del Despliegue: La implementación de estas salvaguardas de bajo nivel ha permitido que la aplicación web de **EVMAudit** opere de manera completamente estable y fluida en la nube. Pese a las severas restricciones del entorno gratuito de Railway, el sistema es capaz de completar con éxito el pipeline completo de auditoría en siete pasos sin registrar caídas en el servicio ni excepciones por falta de recursos.